

Advanced Graph Algorithms — Complete Guide

A comprehensive guide for SDE placements, online assessments, and competitive programming.

1. STRONGLY CONNECTED COMPONENTS (SCC)

1. Overview

A **Strongly Connected Component (SCC)** of a directed graph is a maximal set of vertices such that every vertex in the set is reachable from every other vertex in the same set. In other words, within an SCC, there is a directed path from any vertex to any other vertex.

If you think of a directed graph as a network of one-way streets, an SCC is a group of intersections where you can drive from any intersection to any other while respecting one-way rules.

2. Intuition

Simple Explanation

In a directed graph, some groups of nodes are tightly connected: you can travel from any node in the group to any other node in the group by following directed edges. These groups are SCCs. Once you leave an SCC, you can never come back (otherwise they'd be part of the same SCC).

Analogy

Imagine a group of friends in a party where:

- Each person can pass a message to certain others (directed edges).
- An SCC is a group where a message starting from anyone can reach everyone in the group.
- If you leave this group, you can never get back in.

Step-by-Step Reasoning

1. If you run DFS from a node, you can mark all reachable nodes.
2. But reachability is not symmetric in directed graphs.
3. If we reverse all edges, reachability is also reversed.
4. By combining DFS on the original graph and the reversed graph, we can find SCCs.

Why It Works

The key insight: If you compress each SCC into a single node, the resulting graph is a **DAG** (Directed Acyclic Graph). This DAG (called the condensation graph) is topologically sorted when we process SCCs in reverse finish order of DFS.

3. When to Use It

Use SCC algorithms when:

- **Reachability in directed graphs:** Need to know if nodes are mutually reachable.
- **Cyclic dependency detection:** Finding cycles in dependency graphs (build systems, course prerequisites).
- **Graph condensation:** Reducing a directed graph to a DAG for simpler processing.
- **2-SAT problems:** SCC is a key component of the 2-SAT algorithm.
- **Finding strongly connected subgraphs:** In social networks, web graphs, etc.

Common trigger phrases in problems:

- "strongly connected"
- "all nodes reachable from each other"
- "minimize edges to make graph strongly connected"
- "minimum number of edges to add to make SCC"
- "find the largest group where everyone can message everyone"

4. When Not to Use It

- **Undirected graphs:** Use connected components (DSU/DFS) instead.
- **Single-source reachability:** BFS/DFS is sufficient.
- **Finding cycles in undirected graphs:** Use DFS with parent tracking.
- **DAG processing:** If the graph is already a DAG, SCC compression is unnecessary.
- **Small graphs:** Simple DFS may be more readable.

5. Core Concepts

5.1 Condensation Graph (Component Graph)

The graph formed by contracting each SCC into a single node. It is always a DAG.

Why it matters: Many problems become easier on a DAG — you can use DP, topological sort, etc.

5.2 DFS Finish Time

The time when DFS finishes processing a node (post-order).

Why it matters: Processing nodes in reverse order of finish time gives a topological order of the condensation DAG.

5.3 Transpose Graph (Reversed Graph)

The graph with all edges reversed.

Why it matters: If you can reach v from u in the original graph, you can reach u from v in the reversed graph. This is the key insight for Kosaraju's algorithm.

5.4 Low Link Value

In Tarjan's algorithm, the low-link value of a node is the smallest discovery time reachable from that node (including itself) via DFS tree edges and at most one back edge.

Why it matters: A node is the root of an SCC if its low-link value equals its discovery time.

6. Step-by-Step Algorithm (Kosaraju)

- 1. **First DFS (ordering):** Run DFS on the original graph. Push nodes to a stack in order of finish time (post-order).
- 2. **Reverse the graph:** Create the transpose graph (reverse all edges).
- 3. **Second DFS (extraction):** Pop nodes from the stack one by one. For each unvisited node, run DFS on the reversed graph. All nodes visited in this DFS belong to one SCC.
- 4. **Repeat:** Continue until all nodes are processed.

7. Dry Run

Graph: 0 → 1 → 2 → 0 (cycle), and 1 → 3 → 4 (3 → 4 is a separate path)

Edges: 0→1, 1→2, 2→0, 1→3, 3→4

Step 1: First DFS (ordering)

Step	Node	Action	Stack (finish order)
1	0	Start DFS(0)	
2	1	Visit from 0	
3	2	Visit from 1	
4	0	Already visited (back edge)	
5	2	Finish DFS(2)	[2]
6	3	Visit from 1	[2]
7	4	Visit from 3	[2]
8	4	Finish DFS(4)	[2, 4]
9	3	Finish DFS(3)	[2, 4, 3]
10	1	Finish DFS(1)	[2, 4, 3, 1]
11	0	Finish DFS(0)	[2, 4, 3, 1, 0]

Stack (top to bottom): 0, 1, 3, 4, 2

Step 2: Reverse Graph

Edges reversed: 1→0, 2→1, 0→2, 3→1, 4→3

Step 3: Second DFS on reversed graph

Process nodes in stack order (top to bottom):

Node	DFS on reversed graph	SCC
0	Visits 0, 2, 1 (0→2→1→0 cycle)	SCC1: {0, 1, 2}
3	Visits 3, 4	SCC2: {3, 4}

Result

Two SCCs: {0, 1, 2} and {3, 4}

8. C++ Implementation (Kosaraju)

```
#include <bits/stdc++.h>
using namespace std;

class KosarajuSCC {
    int n;
    vector<vector<int>> adj, radj;
    vector<bool> visited;
    vector<int> order; // finish order stack

    void dfs1(int u) {
        visited[u] = true;
        for (int v : adj[u])
            if (!visited[v])
                dfs1(v);
        order.push_back(u);
    }

    void dfs2(int u, int compId, vector<int>& comp) {
        visited[u] = true;
        comp[u] = compId;
        for (int v : radj[u])
            if (!visited[v])
                dfs2(v, compId, comp);
    }

public:
    KosarajuSCC(int n, vector<vector<int>>& adj)
        : n(n), adj(adj) {
        radj.resize(n);
        for (int u = 0; u < n; u++)
            for (int v : adj[u])
                radj[v].push_back(u);
    }

    // Returns component id for each node (0-indexed)
    vector<int> findSCCs() {
        visited.assign(n, false);
        order.clear();
        for (int i = 0; i < n; i++)
            if (!visited[i])
```

```

        dfs1(i);

        visited.assign(n, false);
        vector<int> comp(n, -1);
        int compId = 0;

        for (int i = n - 1; i >= 0; i--) {
            int u = order[i];
            if (!visited[u])
                dfs2(u, compId++, comp);
        }
        return comp;
    }

    // Build condensation DAG (returns adjacency list of condensed graph)
    vector<vector<int>> buildCondensation(const vector<int>& comp, int compCount)
    {
        vector<vector<int>> cadj(compCount);
        set<pair<int,int>> edges;
        for (int u = 0; u < n; u++) {
            for (int v : adj[u]) {
                int cu = comp[u], cv = comp[v];
                if (cu != cv && edges.find({cu, cv}) == edges.end()) {
                    edges.insert({cu, cv});
                    cadj[cu].push_back(cv);
                }
            }
        }
        return cadj;
    }
};

// Example usage
int main() {
    int n = 5;
    vector<vector<int>> adj = {
        {1},      // 0 -> 1
        {2, 3},   // 1 -> 2, 1 -> 3
        {0},      // 2 -> 0
        {4},      // 3 -> 4
        {}        // 4
    };

    KosarajuSCC scc(n, adj);
    vector<int> comp = scc.findSCCs();

    int compCount = *max_element(comp.begin(), comp.end()) + 1;
    cout << "Number of SCCs: " << compCount << "\n";
    for (int i = 0; i < n; i++)
        cout << "Node " << i << " -> Component " << comp[i] << "\n";

    return 0;
}

```

9. Python Implementation

```
class KosarajuSCC:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.radj = [[] for _ in range(n)]
        for u in range(n):
            for v in adj[u]:
                self.radj[v].append(u)

    def find_sccs(self):
        visited = [False] * self.n
        order = []

        def dfs1(u):
            visited[u] = True
            for v in self.adj[u]:
                if not visited[v]:
                    dfs1(v)
            order.append(u)

        for i in range(self.n):
            if not visited[i]:
                dfs1(i)

        visited = [False] * self.n
        comp = [-1] * self.n
        comp_id = 0

        def dfs2(u, cid):
            visited[u] = True
            comp[u] = cid
            for v in self.radj[u]:
                if not visited[v]:
                    dfs2(v, cid)

        for u in reversed(order):
            if not visited[u]:
                dfs2(u, comp_id)
                comp_id += 1

        return comp, comp_id
```

10. Code Explanation

Kosaraju's Algorithm

First DFS (dfs1):

- Visits all nodes reachable from a starting node.
- After exploring all neighbors, pushes the node to **order** stack.
- This gives us a finish-time ordering: the last finished node is at the end.

Reverse Graph Construction:

- Pre-built in constructor for efficiency.
- `radj[v].push_back(u)` for each edge $u \rightarrow v$.

Second DFS (dfs2):

- Processes nodes in reverse order of finish time.
- On the reversed graph, each DFS finds exactly one SCC.
- Assigns component IDs incrementally.

Condensation Graph (buildCondensation):

- Iterates over all original edges.
- If the two endpoints are in different components, adds an edge between components.
- Uses a set to avoid duplicate edges.

11. Complexity Analysis

Operation	Time	Space
First DFS	$O(V + E)$	$O(V)$
Graph reversal	$O(V + E)$	$O(V + E)$
Second DFS	$O(V + E)$	$O(V)$
Total	$O(V + E)$	$O(V + E)$

- **Best case:** $O(V + E)$ — always linear.
- **Worst case:** $O(V + E)$ — always linear.
- This is optimal: any algorithm must examine all vertices and edges.

12. Common Patterns

Pattern	Description	How to Identify	Example
SCC Compression + DP	Compress SCCs to DAG, then DP on DAG	"maximum path in graph with cycles"	Longest path in directed graph
Minimum edges to make SCC	Find SCCs, count source/sink components	"minimum edges to make strongly connected"	Add edges to make graph SCC
2-SAT	Convert to implication graph, find SCCs	"each variable true/false, constraints"	Boolean satisfiability
Cycle detection in SCC	Any SCC with > 1 node contains a cycle	"find if graph has cycle"	Course schedule II

13. Common Mistakes

- **Forgetting disconnected nodes:** Start DFS from every unvisited node.
- **Not reversing graph correctly:** Make sure all edges are reversed.
- **Mixing up order:** In the second DFS, process nodes in reverse finish order (from end of stack).
- **Stack vs queue:** Use a stack (LIFO) for finish order, not a queue.
- **Off-by-one in component IDs:** Component IDs are 0-indexed in the implementation above.

14. Edge Cases

- **Single node, no edges:** Still one SCC.
- **Empty graph:** Zero SCCs.
- **Already a DAG:** Each node is its own SCC.
- **Complete graph:** One SCC containing all nodes.
- **Self-loops:** A self-loop keeps the node in its own SCC (or merges with others).

15. Variations

Variation	What Changes	When Used	Importance
Kosaraju	Two DFS passes, needs reversed graph	Simple, intuitive	High
Tarjan SCC	Single DFS pass, uses low-link values	No reverse graph needed	High
Kosaraju-Sharir	Same as Kosaraju	Standard textbook version	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Tarjan SCC	Same goal, different approach	Tarjan: single pass, no reverse graph. Kosaraju: more intuitive, two passes.
DSU (Union-Find)	Finds connected components in undirected graphs	DSU is for undirected graphs only. SCC is for directed graphs.
Topological Sort	Works on DAGs; condensation of SCC is a DAG	Use topological sort after SCC compression.
BFS/DFS	Basic reachability	For single-source/single-destination, BFS/DFS is simpler.

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Kosaraju (Basic)	GFG	Find number of SCCs	Easy
Mother Vertex	GFG	Find if there's a vertex that can reach all others	Easy

Medium

Problem	Platform	Idea	Difficulty
Strongly Connected Components	CSES	Find and print SCCs	Medium
Maximum Number of Accepted Invitations	LeetCode	Building to SCC pattern	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Number of Days to Disconnect Island	LeetCode	SCC + articulation	Hard
2-SAT	CSES	SCC-based 2-SAT solver	Hard
Strongly Connected City	Codeforces	SCC on grid graph	Hard

18. Interview Explanation

"Strongly Connected Components partition a directed graph into maximal groups where every node can reach every other node. The key insight is that if you run DFS on the original graph to get finish times, then run DFS on the reversed graph in reverse finish order, each DFS tree in the second pass is exactly one SCC. After compression, SCCs form a DAG. This is useful for problems involving cycles in directed graphs, dependency resolution, and 2-SAT."

19. Revision Notes

- **Goal:** Find maximal strongly connected subgraphs in directed graphs.
- **Kosaraju:** DFS → reverse graph → DFS in reverse finish order.
- **Tarjan:** Single DFS with low-link values.
- **Complexity:** $O(V + E)$ time, $O(V + E)$ space.
- **Key fact:** Condensation graph is always a DAG.
- **Common trap:** Don't forget to start DFS from all unvisited nodes.

20. Final Cheat Sheet

Aspect	Details
When to use	Directed graph, mutual reachability, cycle detection, graph condensation
Main operations	2 DFS passes (Kosaraju) or 1 DFS pass with stack (Tarjan)
Time complexity	$O(V + E)$
Space complexity	$O(V + E)$
Key code idea	<code>order.push_back(u)</code> after DFS; process in reverse on reversed graph
Edge cases	Single node, DAG (each node is its own SCC), complete graph (one SCC)

2. TARJAN SCC ALGORITHM

1. Overview

Tarjan's SCC algorithm finds all strongly connected components in a directed graph using a single DFS pass. Unlike Kosaraju, it does not need a reversed graph. It uses a concept called **low-link values** to identify SCC roots.

2. Intuition

Simple Explanation

During DFS, we assign each node a discovery time (when we first visit it). We also maintain a "low-link" value — the smallest discovery time reachable from this node via DFS tree edges and at most one back edge. When a node's low-link equals its discovery time, it's the root of an SCC.

Analogy

Imagine exploring a cave system with tunnels. You mark each junction with a number (discovery time). You also note the smallest-numbered junction you can reach from your current position. If you are at a junction and the smallest reachable number is your own number, you've found a completely explored chamber (SCC).

Why It Works

The low-link value captures the "earliest" ancestor reachable through the DFS tree plus one back edge. If a node's low-link equals its discovery time, no node in its subtree can reach a node outside the subtree — forming an SCC.

3. When to Use It

- Same situations as Kosaraju.
- Prefer when you don't want to build the reversed graph.
- When you need a single-pass algorithm.
- In competitive programming where execution time is critical (single pass is slightly faster).

4. When Not to Use It

- If you need the condensation DAG edges explicitly (Kosaraju makes this easier).
- For beginners learning SCC (Kosaraju is more intuitive).
- In undirected graphs (use DFS for bridges/articulation points).

5. Core Concepts

5.1 Discovery Time (disc)

The timestamp when a node is first visited during DFS.

5.2 Low-Link Value (low)

The smallest discovery time reachable from the node via:

- Tree edges (edges used in DFS traversal)
- At most one back edge (edge to an ancestor in DFS tree)

5.3 Stack of Nodes

Nodes currently in the DFS recursion stack. A node is part of an SCC until it's popped.

5.4 SCC Root

A node whose $\text{low}[u] == \text{disc}[u]$. This node is the root of its SCC.

6. Step-by-Step Algorithm

1. Initialize $\text{disc}[u] = -1$ (unvisited), $\text{low}[u] = 0$, $\text{inStack}[u] = \text{false}$.
2. Run DFS from each unvisited node: a. Set $\text{disc}[u] = \text{low}[u] = ++\text{time}$. b. Push u onto stack, mark $\text{inStack}[u] = \text{true}$. c. For each neighbor v :
 - If v is unvisited: DFS(v), then set $\text{low}[u] = \min(\text{low}[u], \text{low}[v])$.
 - Else if v is in stack: $\text{low}[u] = \min(\text{low}[u], \text{disc}[v])$ (back edge). d. If $\text{low}[u] == \text{disc}[u]$: pop nodes from stack until u is popped — these form an SCC.

7. Dry Run

Graph: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0, 1 \rightarrow 3 \rightarrow 4$

Step	Node	Action	disc[]	low[]	Stack	SCCs
1	0	Start DFS	disc[0]=1, low[0]=1	[0]		
2	1	From 0 → 1	disc[1]=2, low[1]=2	[0,1]		
3	2	From 1 → 2	disc[2]=3, low[2]=3	[0,1,2]		
4	2	Back edge 2→0	low[2]=min(3,1)=1	[0,1,2]		
5	1	Backtrack, low[1]=min(2,1)=1		[0,1,2]		
6	3	From 1 → 3	disc[3]=4, low[3]=4	[0,1,2,3]		
7	4	From 3 → 4	disc[4]=5, low[4]=5	[0,1,2,3,4]		
8	4	No neighbors, low[4]==disc[4]			Pop 4 → SCC {4}	
9	3	No more neighbors, low[3]==disc[3]			Pop 3 → SCC {3}	

Step	Node	Action	disc[]	low[]	Stack	SCCs
10	1	Backtrack				
11	0	low[0]==disc[0]			Pop 0,1,2 → SCC {0,1,2}	

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class TarjanSCC {
    int n, time;
    vector<vector<int>>> adj;
    vector<int> disc, low, comp;
    vector<bool> inStack;
    stack<int> st;

    void dfs(int u) {
        disc[u] = low[u] = ++time;
        st.push(u);
        inStack[u] = true;

        for (int v : adj[u]) {
            if (disc[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (inStack[v]) {
                low[u] = min(low[u], disc[v]);
            }
        }

        if (low[u] == disc[u]) {
            while (true) {
                int v = st.top(); st.pop();
                inStack[v] = false;
                comp[v] = u; // mark root as component id
                if (v == u) break;
            }
        }
    }

public:
    TarjanSCC(int n, vector<vector<int>>>& adj)
        : n(n), adj(adj), time(0) {
        disc.assign(n, -1);
        low.assign(n, 0);
        comp.assign(n, -1);
        inStack.assign(n, false);
    }
};
```

```

vector<int> findSCCs() {
    for (int i = 0; i < n; i++)
        if (disc[i] == -1)
            dfs(i);
    // Normalize component IDs
    vector<int> compIds(n);
    int id = 0;
    map<int,int> mp;
    for (int i = 0; i < n; i++) {
        if (mp.find(comp[i]) == mp.end())
            mp[comp[i]] = id++;
        compIds[i] = mp[comp[i]];
    }
    return compIds;
}

};

// Example usage
int main() {
    int n = 5;
    vector<vector<int>> adj = {
        {1},      // 0 -> 1
        {2, 3},   // 1 -> 2, 1 -> 3
        {0},      // 2 -> 0
        {4},      // 3 -> 4
        {}        // 4
    };

    TarjanSCC tj(n, adj);
    vector<int> comp = tj.findSCCs();

    int compCount = *max_element(comp.begin(), comp.end()) + 1;
    cout << "Number of SCCs: " << compCount << "\n";
    for (int i = 0; i < n; i++)
        cout << "Node " << i << " -> Component " << comp[i] << "\n";

    return 0;
}

```

9. Python Implementation

```

class TarjanSCC:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.disc = [-1] * n
        self.low = [0] * n
        self.in_stack = [False] * n
        self.comp = [-1] * n
        self.stack = []
        self.time = 0

```

```

def _dfs(self, u):
    self.disc[u] = self.low[u] = self.time
    self.time += 1
    self.stack.append(u)
    self.in_stack[u] = True

    for v in self.adj[u]:
        if self.disc[v] == -1:
            self._dfs(v)
            self.low[u] = min(self.low[u], self.low[v])
        elif self.in_stack[v]:
            self.low[u] = min(self.low[u], self.disc[v])

    if self.low[u] == self.disc[u]:
        while True:
            v = self.stack.pop()
            self.in_stack[v] = False
            self.comp[v] = u
            if v == u:
                break

def find_sccs(self):
    for i in range(self.n):
        if self.disc[i] == -1:
            self._dfs(i)

    # Normalize
    comp_ids = [0] * self.n
    mp = {}
    nid = 0
    for i in range(self.n):
        root = self.comp[i]
        if root not in mp:
            mp[root] = nid
            nid += 1
        comp_ids[i] = mp[root]
    return comp_ids, nid

```

10. Code Explanation

Initialization:

- `disc[]`: discovery time, -1 means unvisited.
- `low[]`: lowest discovery time reachable.
- `inStack[]`: whether node is in current DFS stack.
- `comp[]`: maps each node to its SCC root node.

DFS Function:

- Sets `disc[u] = low[u] = ++time` on entry.
- Pushes `u` onto stack.
- For each neighbor `v`:

- If unvisited: recurse, then `low[u] = min(low[u], low[v])`.
- If in stack (back edge): `low[u] = min(low[u], disc[v])`.
- After exploring all neighbors, if `low[u] == disc[u]`, pop nodes from stack until u — these form an SCC.

Component Normalization:

- The raw `comp[]` maps to root nodes. We normalize to 0, 1, 2, ... IDs.

11. Complexity Analysis

Operation	Time	Space
Single DFS	$O(V + E)$	$O(V)$
Total	$O(V + E)$	$O(V + E)$

Same as Kosaraju, but no need for reversed graph (saves memory).

12. Common Patterns

Same as Kosaraju — see SCC section.

13. Common Mistakes

- **Using `low[v]` instead of `disc[v]` for back edge:** For back edges, use `disc[v]` (not `low[v]`), because `v` might already have a higher low-link.
- **Not checking `inStack`:** When `v` is visited but not in stack, it's part of a completed SCC — ignore it.
- **Stack overflow:** Recursion depth may be $O(V)$ — use iterative DFS or increase stack size for large graphs.

14. Edge Cases

Same as Kosaraju — see SCC section.

15. Variations

Variation	What Changes	When Used	Importance
Tarjan (standard)	Single DFS with low-link	General SCC	High
Gabow's Algorithm	Uses two stacks instead of low-link	Alternative to Tarjan	Low

16. Related Algorithms

- **Kosaraju:** Two-pass algorithm. Easier to understand, but needs reversed graph.
- **Bridges/Articulation Points:** Also use low-link values (undirected graph variant).

17. Practice Problems

Same as SCC section — see Kosaraju practice problems.

18. Interview Explanation

"Tarjan's algorithm finds SCCs in a single DFS pass. Each node gets a discovery time and a low-link value — the smallest discovery time reachable through tree edges and one back edge. When a node's low-link equals its discovery time, it's the root of an SCC. We pop nodes from the stack until we remove the root — these form the SCC. It runs in $O(V+E)$ time and doesn't need a reversed graph."

19. Revision Notes

- **Goal:** Find SCCs in one DFS pass.
- **Key:** $\text{low}[u] = \min(\text{disc}[u], \text{disc}[v] \text{ for back edge, } \text{low}[v] \text{ for tree edge})$.
- **Root condition:** $\text{low}[u] == \text{disc}[u] \rightarrow \text{pop SCC}$.
- **Complexity:** $O(V + E)$ time, $O(V)$ extra space.
- **Trap:** Use $\text{disc}[v]$ (not $\text{low}[v]$) for back edges.

20. Final Cheat Sheet

Aspect	Details
When to use	SCC in directed graphs, single-pass requirement
Main operations	DFS with disc/low tracking and stack
Time complexity	$O(V + E)$
Space complexity	$O(V + E)$
Key code idea	$\text{low}[u] = \min(\text{low}[u], \text{disc}[v])$ for back edges; $\text{low}[u] == \text{disc}[u] \rightarrow \text{pop SCC}$
Edge cases	Node with no edges, already visited not in stack

3. BRIDGES IN A GRAPH

1. Overview

A **bridge** (also called a cut-edge) in an undirected graph is an edge whose removal increases the number of connected components. In other words, bridges are critical edges: if you remove one, the graph becomes disconnected.

2. Intuition

Simple Explanation

In a network of roads, a bridge is a road that, if closed, would split the city into two disconnected parts. There's no alternative route.

Analogy

Imagine a chain of islands connected by bridges. A bridge is critical if destroying it makes it impossible to reach some island. If there's a ferry (alternative path), the bridge is not critical.

Step-by-Step Reasoning

1. Run DFS on the graph.
2. Each edge is either a tree edge (used in DFS) or a back edge (connects to an ancestor).
3. A tree edge (u, v) is a bridge if there is NO back edge from the subtree of v to u or any ancestor of u .
4. Use low-link values to detect this: if $low[v] > disc[u]$, then (u, v) is a bridge.

Why It Works

The low-link value of v tells us the earliest reachable ancestor from the subtree of v . If $low[v] > disc[u]$, it means no node in the subtree of v can reach u or above without going through the edge (u, v) . So removing (u, v) disconnects the subtree.

3. When to Use It

- **Critical connections in networks:** Find which edges are essential for connectivity.
- **Network reliability analysis:** Determine weak points in a network.
- **Finding 2-edge-connected components:** Bridges partition the graph into 2-edge-connected components.
- **Graph vulnerability assessment:** Which edges, if removed, would disconnect the graph.

Common trigger phrases:

- "critical connections"
- "bridges in a graph"
- "edges whose removal disconnects"
- "minimum edges to make graph bridge-free"
- "2-edge-connected components"

4. When Not to Use It

- **Directed graphs:** Bridges are defined for undirected graphs. Use SCC for directed graphs.
- **Finding articulation points:** Different concept (vertices vs edges).
- **Simple connectivity check:** DFS/BFS is enough.
- **Dense graphs with many edges:** Bridges are rare in dense graphs; consider other approaches.

5. Core Concepts

5.1 Tree Edge vs Back Edge (DFS Tree)

In a DFS traversal of an undirected graph:

- **Tree edge:** An edge used to discover a new node (part of DFS tree).
- **Back edge:** An edge connecting a node to an ancestor in the DFS tree.

Why it matters: Only tree edges can be bridges. Back edges provide alternative routes.

5.2 Discovery Time (disc)

The timestamp when a node is first visited.

5.3 Low-Link Value (low)

For an undirected graph, the smallest discovery time reachable from a node via tree edges and **at most one back edge** (excluding the parent edge).

5.4 Bridge Condition

An edge (u, v) is a bridge if `low[v] > disc[u]` (when u is the parent of v in DFS tree).

6. Step-by-Step Algorithm

1. Initialize `disc[u] = -1` for all nodes.
2. Run DFS from any unvisited node: a. Set `disc[u] = low[u] = ++time`. b. For each neighbor v:
 - If v is the parent in DFS tree, skip.
 - If v is unvisited:
 - DFS(v).
 - Set `low[u] = min(low[u], low[v])`.
 - If `low[v] > disc[u]`: edge (u, v) is a bridge.
 - Else (back edge): `low[u] = min(low[u], disc[v])`.

7. Dry Run

Graph: 0 – 1 – 2 – 3 – 0 (square with diagonals... simpler: 0-1-2-0)

Let's take: 0 – 1, 1 – 2, 2 – 0, 1 – 3, 3 – 4

Step	Edge	Action	disc[]	low[]	Bridge?
1	Start at 0		disc[0]=1, low[0]=1		
2	0→1	Tree edge	disc[1]=2, low[1]=2		
3	1→2	Tree edge	disc[2]=3, low[2]=3		
4	2→0	Back edge	low[2]=min(3,1)=1		
5	1→2	Backtrack	low[1]=min(2,1)=1	Check 1-2: low2 > disc1? No	
6	1→3	Tree edge	disc[3]=4, low[3]=4		
7	3→4	Tree edge	disc[4]=5, low[4]=5		
8	4→3	Backtrack	low[4]=5, 5>4? Yes	Bridge: 3-4	
9	3→4	Backtrack	low[3]=min(4,5)=4	Check 1-3: low3 > disc1? No	
10	0→1	Backtrack	low[0]=min(1,1)=1	Check 0-1: low1 > disc0? No	

Bridges found: 3—4

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class BridgeFinder {
    int n, time;
    vector<vector<int>>> adj;
    vector<int> disc, low;
    vector<pair<int,int>>> bridges;

    void dfs(int u, int parent) {
        disc[u] = low[u] = ++time;
        for (int v : adj[u]) {
            if (v == parent) continue;
            if (disc[v] == -1) {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > disc[u]) {
                    bridges.push_back({u, v});
                }
            } else {
                // Back edge
                low[u] = min(low[u], disc[v]);
            }
        }
    }

public:
    BridgeFinder(int n, vector<vector<int>>>& adj)
        : n(n), adj(adj), time(0) {
        disc.assign(n, -1);
        low.assign(n, 0);
    }

    vector<pair<int,int>>> findBridges() {
        bridges.clear();
        for (int i = 0; i < n; i++)
            if (disc[i] == -1)
                dfs(i, -1);
        return bridges;
    }
};

// Example usage
int main() {
    int n = 5;
    vector<vector<int>>> adj = {
        {1, 2}, // 0 connected to 1, 2
        {0, 2, 3}, // 1 connected to 0, 2, 3
        {0, 1}, // 2 connected to 0, 1
        {1, 4}, // 3 connected to 1, 4
        {3} // 4 connected to 3
    };
};

```

```

BridgeFinder bf(n, adj);
auto bridges = bf.findBridges();

cout << "Bridges:\n";
for (auto [u, v] : bridges)
    cout << u << " - " << v << "\n";
// Output: 3 - 4

return 0;
}

```

9. Python Implementation

```

class BridgeFinder:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.disc = [-1] * n
        self.low = [0] * n
        self.time = 0
        self.bridges = []

    def _dfs(self, u, parent):
        self.disc[u] = self.low[u] = self.time
        self.time += 1
        for v in self.adj[u]:
            if v == parent:
                continue
            if self.disc[v] == -1:
                self._dfs(v, u)
                self.low[u] = min(self.low[u], self.low[v])
                if self.low[v] > self.disc[u]:
                    self.bridges.append((u, v))
            else:
                self.low[u] = min(self.low[u], self.disc[v])

    def find_bridges(self):
        for i in range(self.n):
            if self.disc[i] == -1:
                self._dfs(i, -1)
        return self.bridges

```

10. Code Explanation

DFS Function:

- `disc[u] = low[u] = ++time`: Set discovery time.
- For each neighbor `v`:
 - Skip parent to avoid treating the tree edge back as a back edge.

- If v is unvisited: recurse, then update $low[u]$ from child.
- Check bridge condition: $low[v] > disc[u]$.
- If v is visited (back edge): update $low[u] = \min(low[u], disc[v])$.

Bridge Condition:

- $low[v] > disc[u]$ means the subtree of v cannot reach u or above without the edge (u, v) .

11. Complexity Analysis

Operation	Time	Space
DFS traversal	$O(V + E)$	$O(V)$
Total	$O(V + E)$	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Critical connections	Find all bridges in a network	"critical connections in a network"	LeetCode 1192
2-edge-connected components	Remove bridges, remaining components are 2-edge-connected	"bridge-free components"	CSES
Minimum edges to make graph bridge-free	Add minimum edges to eliminate all bridges	"make graph 2-edge-connected"	Codeforces

13. Common Mistakes

- **Not skipping parent:** The parent edge is a tree edge; treating it as a back edge would cause wrong low values.
- **Using $low[v]$ instead of $disc[v]$ for back edges:** For undirected bridges, use $disc[v]$ for back edges.
- **Forgetting multiple edges:** If there are multiple edges between same nodes, those edges are not bridges (there's an alternative route).
- **Disconnected graph:** Start DFS from each unvisited node.

14. Edge Cases

- **Single node, no edges:** Zero bridges.
- **Two nodes, one edge:** That edge is a bridge.
- **Cycle:** No bridges in a simple cycle.
- **Complete graph ($K_n, n \geq 3$):** No bridges.
- **Tree (no cycles):** Every edge is a bridge.
- **Multiple edges between same nodes:** Those edges are not bridges.

15. Variations

Variation	What Changes	When Used	Importance
Bridges in directed graph	Use SCC concepts	Rarely asked	Low
Bridge tree	Compress 2-edge-connected components into tree	After finding bridges	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Articulation Points	Same low-link concept, different condition	Bridges → edges, Articulation → vertices
SCC (Tarjan/Kosaraju)	Different concept (directed vs undirected)	Bridges for undirected, SCC for directed
DSU	Can find bridges in special cases	DSU is simpler but can't find all bridges

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Find Bridges in a Graph	GFG	Basic bridge detection	Easy
Critical Connections	LeetCode 1192	Find all bridges in network	Easy

Medium

Problem	Platform	Idea	Difficulty
Critical Edges	Codeforces	Bridges in a graph	Medium
Bridge Tree	Codeforces	Build bridge tree from bridges	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Number of Days to Disconnect Island	LeetCode	Bridges + articulation	Hard
2-Edge-Connected Components	CSES	SCC-like for bridges	Hard

18. Interview Explanation

"A bridge is an edge whose removal increases the number of connected components. We find bridges using DFS with low-link values. For each node, we track the discovery time and the smallest discovery

time reachable from its subtree. An edge (u, v) is a bridge if $low[v] > disc[u]$, meaning the subtree of v cannot reach u or above without that edge. This runs in $O(V+E)$ time."

19. Revision Notes

- **Bridge condition:** $low[v] > disc[u]$ (parent u, child v).
- **Low-link:** smallest discovery time reachable via tree edges + one back edge.
- **Skip parent edge** to avoid false back edge.
- **Complexity:** $O(V + E)$.
- **Trap:** For multiple edges, they are not bridges.

20. Final Cheat Sheet

Aspect	Details
When to use	Critical connections, network reliability, 2-edge-connected components
Main operations	DFS with low-link values
Time complexity	$O(V + E)$
Space complexity	$O(V)$
Key code idea	<code>if (low[v] > disc[u]) → bridge(u,v)</code>
Edge cases	Single node, cycle (no bridges), tree (all edges are bridges)

4. ARTICULATION POINTS

1. Overview

An **articulation point** (also called a cut-vertex) is a vertex in an undirected graph whose removal increases the number of connected components. These are the critical nodes that, if removed, disconnect the graph.

2. Intuition

Simple Explanation

In a network, an articulation point is a person whose removal would break communication between others. If this person leaves, some people can no longer reach each other.

Analogy

Think of a city's road network. An intersection is an articulation point if closing it would make it impossible to travel between some parts of the city.

Step-by-Step Reasoning

1. Run DFS on the graph.
2. Two cases for a node being an articulation point:

- **Root of DFS tree:** It's an articulation point if it has ≥ 2 children in the DFS tree.
- **Non-root node u :** It's an articulation point if there exists a child v such that $\text{low}[v] \geq \text{disc}[u]$ (no back edge from subtree of v to above u).

Why It Works

For a non-root node u , if $\text{low}[v] \geq \text{disc}[u]$, the subtree of v has no back edge to any ancestor of u . Removing u disconnects this subtree from the rest of the graph.

3. When to Use It

- **Critical nodes in networks:** Find which nodes are essential for connectivity.
- **Network vulnerability:** Identify single points of failure.
- **Finding 2-vertex-connected components:** Partition graph by articulation points.
- **Graph decomposition:** Split graph at articulation points into biconnected components.

Common trigger phrases:

- "critical nodes"
- "articulation points"
- "cut vertices"
- "vertices whose removal disconnects"
- "single points of failure"

4. When Not to Use It

- **Directed graphs:** Use different concepts (not directly applicable).
- **Finding bridges:** Different concept (edges vs vertices).
- **Simple connectivity:** BFS/DFS is enough.
- **Dense graphs:** Articulation points are rare; consider if this analysis is relevant.

5. Core Concepts

5.1 DFS Tree Root

If the root of the DFS tree has more than one child (in the DFS tree), it's an articulation point. Removing the root separates these subtrees.

5.2 Low-Link Value

Same as bridges: the smallest discovery time reachable via tree edges and one back edge.

5.3 Articulation Condition (Non-Root)

$\text{low}[v] \geq \text{disc}[u]$ for some child v of u . Note the difference from bridges: $\geq$ instead of $>$.

5.4 Biconnected Components

Maximal 2-vertex-connected subgraphs. Articulation points are the boundaries between biconnected components.

6. Step-by-Step Algorithm

1. Initialize $disc[u] = -1$ for all nodes.
2. Run DFS from each unvisited node:
 - a. Set $disc[u] = low[u] = ++time$.
 - b. Count children.
 - c. For each neighbor v :
 - If v is parent, skip.
 - If v is unvisited:
 - DFS(v).
 - Increment children count.
 - $low[u] = \min(low[u], low[v])$.
 - If u is not root and $low[v] \geq disc[u]$: u is articulation point.
 - Else (back edge): $low[u] = \min(low[u], disc[v])$.
 - d. If u is root and children > 1 : u is articulation point.

7. Dry Run

Graph: $0 - 1 - 2 - 3, 1 - 4 - 5$

```

0 - 1 - 2 - 3
  |
  4 - 5

```

DFS from 0: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{back} \rightarrow 1 \rightarrow 4 \rightarrow 5$

Step	Node	Action	disc[]	low[]	Children	Art. Point?
1	0	Start, root	disc[0]=1, low[0]=1	0		
2	1	Tree edge $0 \rightarrow 1$	disc[1]=2, low[1]=2	0		
3	2	Tree edge $1 \rightarrow 2$	disc[2]=3, low[2]=3	0		
4	3	Tree edge $2 \rightarrow 3$	disc[3]=4, low[3]=4	0		
5	3	No more neighbors			low[3]=4, disc[2]=3, $4 \geq 3$? Yes $\rightarrow$ 2 is art. point	
6	2	Backtrack	low[2]=min(3,4)=3		Check child 3: low[3]=4 $\geq$ disc[2]=3 $\rightarrow$ 2 is AP	
7	1	$2 \rightarrow 1$: back edge? No, 2's parent is 1				

Step	Node	Action	disc[]	low[]	Children	Art. Point?
		1→4: Tree edge	disc[4]=5, low[4]=5			
8	5	Tree edge 4→5	disc[5]=6, low[5]=6			
9	5	Backtrack			low[5]=6, disc[4]=5, 6≥5? Yes → 4 is AP	
10	4	Backtrack	low[4]=min(5,6)=5		Check child 5: low[5]=6 ≥ disc[4]=5 → 4 is AP	
11	1	Backtrack	low[1]=min(2,5)=2		Check child 2: low[2]=3 ≥ disc[1]=2 → 1 is AP	
					Check child 4: low[4]=5 ≥ disc[1]=2 → 1 is AP	
12	0	Root		children=1	0 has 1 child, not AP	

Articulation points: 1, 2, 4

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class ArticulationPointFinder {
    int n, time;
    vector<vector<int>>> adj;
    vector<int> disc, low;
    vector<bool> isAP;
    set<int> articulationPoints;

    void dfs(int u, int parent) {
        disc[u] = low[u] = ++time;
        int children = 0;

        for (int v : adj[u]) {
            if (v == parent) continue;
            if (disc[v] == -1) {
                children++;
                dfs(v, u);
                low[u] = min(low[u], low[v]);

                // Non-root case
```

```

        if (parent != -1 && low[v] >= disc[u]) {
            articulationPoints.insert(u);
        }
    } else {
        // Back edge
        low[u] = min(low[u], disc[v]);
    }
}

// Root case
if (parent == -1 && children > 1) {
    articulationPoints.insert(u);
}
}

public:
    ArticulationPointFinder(int n, vector<vector<int>>& adj)
        : n(n), adj(adj), time(0) {
        disc.assign(n, -1);
        low.assign(n, 0);
        isAP.assign(n, false);
    }

    set<int> findArticulationPoints() {
        articulationPoints.clear();
        for (int i = 0; i < n; i++)
            if (disc[i] == -1)
                dfs(i, -1);
        return articulationPoints;
    }
};

// Example usage
int main() {
    int n = 6;
    vector<vector<int>> adj = {
        {1},           // 0
        {0, 2, 4},     // 1
        {1, 3},        // 2
        {2},           // 3
        {1, 5},        // 4
        {4}            // 5
    };

    ArticulationPointFinder apf(n, adj);
    auto points = apf.findArticulationPoints();

    cout << "Articulation Points: ";
    for (int p : points) cout << p << " ";
    cout << "\n";

    return 0;
}

```

9. Python Implementation

```
class ArticulationPointFinder:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.disc = [-1] * n
        self.low = [0] * n
        self.time = 0
        self.art_points = set()

    def _dfs(self, u, parent):
        self.disc[u] = self.low[u] = self.time
        self.time += 1
        children = 0

        for v in self.adj[u]:
            if v == parent:
                continue
            if self.disc[v] == -1:
                children += 1
                self._dfs(v, u)
                self.low[u] = min(self.low[u], self.low[v])

                if parent != -1 and self.low[v] >= self.disc[u]:
                    self.art_points.add(u)
            else:
                self.low[u] = min(self.low[u], self.disc[v])

        if parent == -1 and children > 1:
            self.art_points.add(u)

    def find_articulation_points(self):
        for i in range(self.n):
            if self.disc[i] == -1:
                self._dfs(i, -1)
        return self.art_points
```

10. Code Explanation

DFS Function:

- `disc[u] = low[u] = ++time`: Standard initialization.
- `children` counter: Only meaningful for root.
- For each neighbor `v`:
 - Skip parent.
 - If unvisited: recurse, update low, check `low[v] >= disc[u]` for non-root.
 - If visited: back edge, update low.
- Root check: if `parent == -1 && children > 1`, root is AP.

Key difference from Bridges:

- Bridges: $\text{low}[v] > \text{disc}[u]$ (strict inequality).
- Articulation points: $\text{low}[v] \geq \text{disc}[u]$ (with equality allowed).

11. Complexity Analysis

Operation	Time	Space
DFS traversal	$O(V + E)$	$O(V)$
Total	$O(V + E)$	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Critical nodes	Find all articulation points	"critical nodes in network"	GFG
Biconnected components	Partition by articulation points	"biconnected components"	Codeforces
Network reliability	Find single points of failure	"network vulnerability"	Various

13. Common Mistakes

- **Not handling root separately:** Root has different condition ($\text{children} > 1$).
- **Using $>$ instead of $\geq$:** For articulation points, use $\geq$ (not $>$).
- **Not skipping parent:** Same as bridges.
- **Counting total children instead of DFS tree children:** Only count children in the DFS tree.

14. Edge Cases

- **Single node:** Not an articulation point (no edges to disconnect).
- **Two nodes, one edge:** Neither is an articulation point (removing one leaves one node, which is technically one component).
- **Path graph:** All internal nodes are articulation points.
- **Cycle:** No articulation points.
- **Complete graph ($K_n, n \geq 3$):** No articulation points.

15. Variations

Variation	What Changes	When Used	Importance
Biconnected Components	Find maximal 2-vertex-connected subgraphs	Graph decomposition	Medium
Block Cut Tree	Tree connecting articulation points and biconnected components	Advanced graph analysis	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Bridges	Same low-link concept, different condition	> for bridges, >= for articulation points
SCC	Different concept (directed)	Bridges/AP for undirected, SCC for directed
Biconnected Components	Built on top of articulation points	After finding APs, partition graph

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Articulation Points	GFG	Basic AP detection	Easy
Critical Routers	LeetCode	Find articulation points in network	Easy

Medium

Problem	Platform	Idea	Difficulty
Biconnected Graph	Codeforces	Check if graph has no articulation points	Medium
Find Bridges and Articulation Points	HackerEarth	Combined detection	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Number of Days to Disconnect Island	LeetCode 1568	Bridges + articulation points on grid	Hard
Block Cut Tree Construction	Codeforces	Build block cut tree from APs	Hard

18. Interview Explanation

"An articulation point is a vertex whose removal disconnects the graph. We find them using DFS with low-link values. There are two cases: the root is an articulation point if it has more than one child in the DFS tree. A non-root node u is an articulation point if there exists a child v such that $low[v] \geq disc[u]$, meaning the subtree of v cannot reach above u without passing through u . This runs in $O(V+E)$ time."

19. Revision Notes

- **AP condition (non-root):** $low[child] \geq disc[u]$.
- **AP condition (root):** $children > 1$.
- **Low-link:** smallest disc reachable via tree edges + one back edge.

- **Complexity:** $O(V + E)$.
- **Trap:** Use $\geq$ not $>$; handle root separately.

20. Final Cheat Sheet

Aspect	Details
When to use	Critical nodes, network vulnerability, biconnected components
Main operations	DFS with low-link, root check
Time complexity	$O(V + E)$
Space complexity	$O(V)$
Key code idea	<code>if (parent != -1 && low[v] >= disc[u]) → AP; root: if (children > 1) → AP</code>
Edge cases	Single node (not AP), path (internal nodes are APs), cycle (no APs)

5. EULER PATH / CIRCUIT

1. Overview

An **Euler path** (Eulerian trail) is a path in a graph that visits every edge exactly once. An **Euler circuit** (Eulerian cycle) is an Euler path that starts and ends at the same vertex.

The problem of finding such paths is one of the oldest in graph theory, dating back to Euler's 1736 solution to the "Seven Bridges of Königsberg" problem.

2. Intuition

Simple Explanation

Can you draw a figure without lifting your pen and without tracing any line twice? That's an Euler path. If you can also end where you started, it's an Euler circuit.

Analogy

Imagine a postman delivering mail on every street in a neighborhood. The postman wants to walk each street exactly once. An Euler path gives the route. If the postman can also return to the starting point, it's an Euler circuit.

Step-by-Step Reasoning

1. For an Euler circuit: every vertex must have even degree (each time you enter, you must leave).
2. For an Euler path: exactly 0 or 2 vertices can have odd degree (start and end).
3. The graph must be connected (ignoring isolated vertices with degree 0).

4. To find the actual path: use Hierholzer's algorithm — start from a valid start vertex, traverse edges, removing them, and when stuck, backtrack and add the vertex to the circuit.

Why It Works

Hierholzer's algorithm: When you traverse edges, you might create a cycle that doesn't cover all edges. By backtracking and inserting sub-tours, you eventually cover all edges. The algorithm ensures each edge is used exactly once.

3. When to Use It

- **Finding Euler path/circuit in a graph:** The classic application.
- **DNA fragment assembly:** Eulerian paths in de Bruijn graphs.
- **Route planning:** Find a route that uses every road exactly once.
- **Circuit design:** Testing all connections in a circuit.
- **Chinese Postman Problem:** Find shortest route that traverses every edge at least once.

Common trigger phrases:

- "Eulerian path/circuit"
- "visit every edge exactly once"
- "draw without lifting pen"
- "rearrange words to form a chain where last letter of one equals first of next"
- "de Bruijn sequence"

4. When Not to Use It

- **Hamiltonian path:** Visits every vertex exactly once (NP-complete, much harder).
- **Shortest path:** Dijkstra/BFS for shortest routes.
- **Minimum spanning tree:** Different objective.
- **Graph with more than 2 odd-degree vertices:** No Euler path exists.
- **Disconnected graph with edges in multiple components:** No Euler path exists.

5. Core Concepts

5.1 Euler Path vs Euler Circuit

- **Euler circuit:** Starts and ends at same vertex, uses every edge once.
- **Euler path:** Uses every edge once, may start and end at different vertices.

5.2 Degree Condition

- **Euler circuit:** All vertices have even degree.
- **Euler path:** Exactly 0 or 2 vertices have odd degree.
- **No Euler path:** More than 2 vertices have odd degree.

5.3 Connectivity Condition

All non-zero degree vertices must belong to a single connected component (ignoring isolated vertices).

5.4 Hierholzer's Algorithm

The standard algorithm for finding an Euler path/circuit. It uses a stack-based approach to merge cycles.

6. Step-by-Step Algorithm (Hierholzer)

1. Check conditions:

- Count vertices with odd degree.
- If odd count > 2: no Euler path.
- If odd count == 0: Euler circuit exists (start anywhere).
- If odd count == 2: Euler path exists (start at one odd-degree vertex, end at other).

2. Choose start vertex:

- Circuit: any vertex with degree > 0.
- Path: one of the odd-degree vertices.

3. **Traverse:** a. Start at **u**, follow unused edges, removing them as you go. b. When stuck (no unused edges from current vertex), push vertex to circuit stack. c. Backtrack: pop previous vertex from path stack and continue.

4. **Reverse the circuit stack** to get the Euler path/circuit.

7. Dry Run

Graph: 0 — 1 — 2 — 0 — 3 — 4 — 2 — 3 (edges: 0-1, 1-2, 2-0, 0-3, 3-4, 4-2, 2-3)

Degrees: 0:3, 1:2, 2:4, 3:3, 4:2 Odd degree vertices: 0, 3 (exactly 2) → Euler path exists.

Start at 0 (odd degree).

Step	Current	Action	Edge(s) Remaining	Circuit Stack
1	0	Start	all	
2	0→1	Remove 0-1	1-2, 2-0, 0-3, 3-4, 4-2, 2-3	
3	1→2	Remove 1-2	2-0, 0-3, 3-4, 4-2, 2-3	
4	2→0	Remove 2-0	0-3, 3-4, 4-2, 2-3	
5	0→3	Remove 0-3	3-4, 4-2, 2-3	
6	3→4	Remove 3-4	4-2, 2-3	
7	4→2	Remove 4-2	2-3	
8	2→3	Remove 2-3	none	
9	3	Stuck, push 3		[3]
10	2	Stuck (no edges), push 2		[3, 2]
11	4	Stuck, push 4		[3, 2, 4]

Step	Current	Action	Edge(s) Remaining	Circuit Stack
12	3	Stuck, push 3		[3, 2, 4, 3]
13	0	Stuck, push 0		[3, 2, 4, 3, 0]
14	2	Stuck, push 2		[3, 2, 4, 3, 0, 2]
15	1	Stuck, push 1		[3, 2, 4, 3, 0, 2, 1]
16	0	Stuck, push 0		[3, 2, 4, 3, 0, 2, 1, 0]

Reversed: 0 → 1 → 2 → 0 → 3 → 4 → 2 → 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class EulerPathFinder {
    int n;
    vector<unordered_multiset<int>> adj;
    vector<pair<int,int>> edges;

public:
    EulerPathFinder(int n, vector<pair<int,int>>& edges)
        : n(n), edges(edges) {
        adj.resize(n);
        for (auto [u, v] : edges) {
            adj[u].insert(v);
            adj[v].insert(u);
        }
    }

    // Returns Euler path/circuit, or empty vector if none exists
    vector<int> findEulerPath() {
        // Count odd degree vertices
        vector<int> odd;
        for (int i = 0; i < n; i++) {
            if (adj[i].size() % 2 == 1)
                odd.push_back(i);
        }

        if (odd.size() > 2) return {}; // No Euler path

        // Check connectivity (ignoring isolated vertices)
        int start = 0;
        for (int i = 0; i < n; i++) {
            if (adj[i].size() > 0) { start = i; break; }
        }

        vector<bool> visited(n, false);
        queue<int> q;
```

```

q.push(start);
visited[start] = true;
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) {
        if (!visited[v]) {
            visited[v] = true;
            q.push(v);
        }
    }
}
for (int i = 0; i < n; i++) {
    if (adj[i].size() > 0 && !visited[i])
        return {}; // Disconnected
}

// Choose start vertex
int startVertex = odd.empty() ? start : odd[0];

// Hierholzer's algorithm
vector<int> circuit;
stack<int> st;
st.push(startVertex);

// Use local copy of adjacency
vector<unordered_multiset<int>> tempAdj = adj;

while (!st.empty()) {
    int u = st.top();
    if (tempAdj[u].empty()) {
        circuit.push_back(u);
        st.pop();
    } else {
        int v = *tempAdj[u].begin();
        tempAdj[u].erase(tempAdj[u].find(v));
        tempAdj[v].erase(tempAdj[v].find(u));
        st.push(v);
    }
}

// Check if all edges are used
for (int i = 0; i < n; i++) {
    if (!tempAdj[i].empty()) return {};
}

reverse(circuit.begin(), circuit.end());
return circuit;
}
};

// Example usage
int main() {
    vector<pair<int,int>> edges = {
        {0, 1}, {1, 2}, {2, 0}, {0, 3}, {3, 4}, {4, 2}, {2, 3}
    };
}

```

```

};
int n = 5;

EulerPathFinder epf(n, edges);
vector<int> path = epf.findEulerPath();

if (path.empty()) {
    cout << "No Euler path exists\n";
} else {
    cout << "Euler path: ";
    for (int v : path) cout << v << " ";
    cout << "\n";
}

return 0;
}

```

9. Python Implementation

```

from collections import defaultdict, deque

class EulerPathFinder:
    def __init__(self, n, edges):
        self.n = n
        self.adj = defaultdict(list)
        for u, v in edges:
            self.adj[u].append(v)
            self.adj[v].append(u)

    def find_euler_path(self):
        # Count odd degree vertices
        odd = [v for v in range(self.n) if len(self.adj[v]) % 2 == 1]

        if len(odd) > 2:
            return [] # No Euler path

        # Find start vertex
        start = 0
        for v in range(self.n):
            if len(self.adj[v]) > 0:
                start = v
                break

        # Check connectivity
        visited = [False] * self.n
        q = deque([start])
        visited[start] = True
        while q:
            u = q.popleft()
            for v in self.adj[u]:
                if not visited[v]:

```

```

        visited[v] = True
        q.append(v)

    for v in range(self.n):
        if len(self.adj[v]) > 0 and not visited[v]:
            return [] # Disconnected

    start_vertex = odd[0] if odd else start

    # Hierholzer's algorithm
    # Use a local copy of adjacency (list of lists)
    temp_adj = {v: list(neighbors) for v, neighbors in self.adj.items()}

    circuit = []
    stack = [start_vertex]

    while stack:
        u = stack[-1]
        if not temp_adj.get(u, []):
            circuit.append(u)
            stack.pop()
        else:
            v = temp_adj[u].pop()
            temp_adj[v].remove(u)
            stack.append(v)

    # Check all edges used
    for v in temp_adj:
        if temp_adj[v]:
            return []

    circuit.reverse()
    return circuit

```

10. Code Explanation

Finding Start Vertex:

- For Euler circuit: any vertex with degree > 0.
- For Euler path: one of the odd-degree vertices.

Hierholzer's Algorithm:

- Uses a stack to track current path.
- While at vertex **u**:
 - If no unused edges from **u**: we're stuck → add **u** to circuit.
 - Else: take any unused edge (**u**, **v**), remove it, and move to **v**.
- At the end, reverse the circuit to get the correct order.

Edge Removal:

- We use `unordered_multiset` for efficient removal of individual edges.

- In Python, we use lists and remove elements.

11. Complexity Analysis

Operation	Time	Space
Degree check	$O(V)$	$O(1)$
Connectivity check	$O(V + E)$	$O(V)$
Hierholzer traversal	$O(E)$	$O(V + E)$
Total	$O(V + E)$	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Eulerian circuit existence	Check if all vertices have even degree + connectivity	"does Euler circuit exist?"	Codeforces
Eulerian path existence	Check if 0 or 2 odd-degree vertices + connectivity	"does Euler path exist?"	GFG
Word chain (rearrange words)	Build graph of first/last letters, find Euler path	"last letter of one equals first of next"	LeetCode
De Bruijn sequence	Euler circuit in de Bruijn graph	"shortest string containing all substrings"	CSES

13. Common Mistakes

- **Forgetting connectivity check:** Even if degree conditions are met, graph must be connected (ignoring isolated vertices).
- **Not handling directed graphs:** Directed Euler path requires different conditions (in-degree = out-degree for each vertex).
- **Removing wrong edge:** When using multisets, be careful to remove the correct occurrence.
- **Not checking if all edges are used:** The algorithm might leave a cycle if the graph has multiple components with edges.

14. Edge Cases

- **Single vertex, no edges:** Trivial circuit (empty path).
- **Single edge (two vertices):** Euler path exists.
- **All vertices isolated:** No edges, trivially satisfies conditions.
- **Graph with exactly 2 odd-degree vertices:** Euler path exists between them.
- **Graph with 4 odd-degree vertices:** No Euler path.
- **Complete graph with odd number of vertices:** Euler circuit exists (all degrees are even? K_n : each vertex has degree $n-1$; if n is odd, $n-1$ is even $\rightarrow$ circuit exists).

15. Variations

Variation	What Changes	When Used	Importance
Directed Euler Path	Check in-degree == out-degree for circuit	Directed graphs	High
Chinese Postman Problem	Find shortest path visiting all edges (can repeat)	Euler path doesn't exist, need to add edges	Medium
De Bruijn Sequence	Euler circuit in de Bruijn graph	Bioinformatics, combinatorics	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Hamiltonian Path	Visits each vertex once (NP-complete)	Euler = edges, Hamiltonian = vertices
DFS	Hierholzer is based on DFS	DFS for basic traversal, Hierholzer for Euler path
Topological Sort	For DAGs	Euler path is for undirected/directed graphs

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Eulerian Path in an Undirected Graph	GFG	Check if Euler path exists	Easy
Find the Euler Circuit in a Graph	GFG	Find Euler circuit	Easy

Medium

Problem	Platform	Idea	Difficulty
Reconstruct Itinerary	LeetCode 332	Euler path in directed graph	Medium
Cracking the Code	LeetCode 753	De Bruijn sequence (Euler circuit)	Medium

Hard

Problem	Platform	Idea	Difficulty
Eulerian Path	CSES	Find Euler path in directed graph	Hard
Chinese Postman Problem	Codeforces	Minimum cost to traverse all edges	Hard

18. Interview Explanation

"An Euler path visits every edge exactly once. For an undirected graph, an Euler circuit exists if all vertices have even degree and the graph is connected. An Euler path exists if exactly 0 or 2 vertices have odd degree. To find the actual path, we use Hierholzer's algorithm: we traverse edges, removing

them, and when stuck, we backtrack and add the vertex to the circuit. The algorithm runs in $O(V+E)$ time."

19. Revision Notes

- **Euler circuit:** all vertices even degree + connected.
- **Euler path:** 0 or 2 odd-degree vertices + connected.
- **Hierholzer:** DFS traversal, remove edges as used, backtrack when stuck.
- **Complexity:** $O(V + E)$.
- **Trap:** Don't forget connectivity check; handle directed case separately.

20. Final Cheat Sheet

Aspect	Details
When to use	Visit every edge exactly once, word chains, de Bruijn sequences
Main operations	Degree check, connectivity check, Hierholzer traversal
Time complexity	$O(V + E)$
Space complexity	$O(V + E)$
Key code idea	Stack-based DFS, remove edges as used, reverse at end
Edge cases	Single vertex, disconnected with edges, more than 2 odd-degree vertices

6. BIPARTITE MATCHING

1. Overview

Bipartite matching is the problem of finding a maximum set of edges in a bipartite graph such that no two edges share a vertex. A bipartite graph is one where vertices can be divided into two disjoint sets (U and V), and all edges go between U and V.

This is equivalent to finding the maximum number of pairs we can form from two groups, where each person can only be paired with specific compatible partners.

2. Intuition

Simple Explanation

You have two groups of items (e.g., jobs and applicants). Each applicant is qualified for certain jobs. You want to assign as many applicants as possible to distinct jobs, with each job getting at most one applicant. This is maximum bipartite matching.

Analogy

Think of a dating app with two groups: mentors and mentees. Each mentor can only work with certain mentees. You want to pair up as many mentor-mentee pairs as possible, with each person in at most one pair.

Step-by-Step Reasoning

1. Start with an empty matching.
2. Try to find an augmenting path: a path that starts at an unmatched vertex in U , ends at an unmatched vertex in V , and alternates between non-matching and matching edges.
3. If such a path exists, flip the edges along it (non-matching $\rightarrow$ matching, matching $\rightarrow$ non-matching), which increases the matching size by 1.
4. Repeat until no augmenting path exists.

Why It Works

Berge's Lemma: A matching is maximum if and only if there is no augmenting path. By repeatedly finding and flipping augmenting paths, we increase the matching size until it's maximum.

3. When to Use It

- **Assignment problems:** Assign tasks to workers, jobs to machines, etc.
- **Pairing problems:** Form as many valid pairs as possible.
- **Minimum vertex cover in bipartite graphs:** König's theorem: size of minimum vertex cover = size of maximum matching.
- **Maximum independent set in bipartite graphs:** Total vertices - matching size.
- **Hall's marriage theorem:** Check if perfect matching exists.
- **Grid/board problems:** Where you can pair cells (e.g., domino placement).

Common trigger phrases:

- "maximum matching"
- "assign/assignments"
- "pair up"
- "maximum bipartite matching"
- "minimum vertex cover"
- "Hall's theorem"

4. When Not to Use It

- **General graph matching:** Use Blossom algorithm (Edmonds), much more complex.
- **Maximum flow directly:** Bipartite matching is a special case of max flow. Use max flow for more complex constraints.
- **Weighted matching:** Use Hungarian algorithm for maximum weight matching.
- **Stable matching:** Gale-Shapley algorithm for stable matching.

5. Core Concepts

5.1 Bipartite Graph

A graph where vertices can be partitioned into two sets U and V , with all edges going between U and V (no edges within U or V).

5.2 Matching

A set of edges with no common vertices. Size = number of edges in the matching.

5.3 Augmenting Path

A path that starts at an unmatched vertex in U , ends at an unmatched vertex in V , and alternates between non-matching and matching edges.

5.4 Alternating Path

A path that alternates between non-matching and matching edges (not necessarily starting/ending at unmatched vertices).

5.5 Perfect Matching

A matching that covers all vertices (size = $|U| = |V|$).

6. Step-by-Step Algorithm (DFS-based Augmenting Path)

1. Build adjacency list for U -side vertices.
2. For each vertex u in U : a. Try to find an augmenting path starting from u . b. Use DFS: for each neighbor v of u :
 - If v is already visited in this attempt, skip.
 - Mark v as visited.
 - If v is unmatched or the vertex matched to v can be reassigned (DFS on $\text{match}[v]$), then:
 - Set $\text{match}[v] = u$.
 - Return true (augmenting path found).
- c. If DFS returns true, matching size increases by 1.

7. Dry Run

$U = \{0, 1, 2\}$, $V = \{3, 4, 5\}$ Edges: 0-3, 0-4, 1-3, 1-5, 2-4

Step	u	DFS	match[]	Matching Size
1	0	0-3 (unmatched) → $\text{match}[3]=0$	$\text{match}[3]=0$	1
2	1	1-3 (matched w/0), try 0: 0-4 (unmatched) → rematch	$\text{match}[3]=1, \text{match}[4]=0$	2
3	2	2-4 (matched w/0), try 0: 0-3 (matched w/1), try 1: 1-5 (unmatched) → rematch	$\text{match}[3]=1, \text{match}[4]=2, \text{match}[5]=1$	3

Final matching: (0-3, 1-5, 2-4) or (0-4, 1-3, 2-4)... Let's trace more carefully.

Actually, let's trace step by step:

Initial: $\text{match}[3]=\text{match}[4]=\text{match}[5]=-1$, matching size = 0

u=0: $\text{visited}[3]=\text{false}, \text{visited}[4]=\text{false}$

- DFS(0): try neighbor 3
 - $v=3, \text{match}[3]=-1 \rightarrow \text{set } \text{match}[3]=0$, return true

- match[3]=0, size=1

u=1: visited[3]=false, visited[5]=false

- DFS(1): try neighbor 3
 - v=3, match[3]=0 → DFS(match[3]=0, looking for alternate)
 - visited[3]=true
 - DFS(0): try neighbors 3 (visited), 4
 - v=4, match[4]=-1 → set match[4]=0, return true
 - Back to DFS(1): set match[3]=1, return true
- match[3]=1, match[4]=0, size=2

u=2: visited[4]=false

- DFS(2): try neighbor 4
 - v=4, match[4]=0 → DFS(match[4]=0, looking for alternate)
 - visited[4]=true
 - DFS(0): try neighbors 3 (match[3]=1, not visited[3]), 4 (visited)
 - DFS(0): try neighbor 3
 - v=3, match[3]=1 → DFS(match[3]=1)
 - visited[3]=true
 - DFS(1): try neighbors 3 (visited), 5
 - v=5, match[5]=-1 → set match[5]=1, return true
 - Back to DFS(0): set match[3]=0, return true
 - Back to DFS(2): set match[4]=2, return true
 - match[3]=0, match[4]=2, match[5]=1, size=3

Final matching: (0-3, 1-5, 2-4), size = 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class BipartiteMatching {
    int n, m; // sizes of U and V
    vector<vector<int>> adj; // adjacency from U to V
    vector<int> matchV; // which U vertex is matched to each V vertex
    vector<int> matchU; // which V vertex is matched to each U vertex
    vector<int> visited; // visited in current DFS attempt
    int iter;

    bool dfs(int u) {
        for (int v : adj[u]) {
            if (visited[v] == iter) continue;
            visited[v] = iter;
            if (matchV[v] == -1 || dfs(matchV[v])) {
                matchV[v] = u;
                matchU[u] = v;
                return true;
            }
        }
        return false;
    }
};
```

```

    }
    }
    return false;
}

public:
    BipartiteMatching(int n, int m, vector<vector<int>>& adj)
        : n(n), m(m), adj(adj) {
        matchV.assign(m, -1);
        matchU.assign(n, -1);
        visited.assign(m, 0);
        iter = 0;
    }

    int maxMatching() {
        int matching = 0;
        for (int u = 0; u < n; u++) {
            iter++;
            if (dfs(u)) {
                matching++;
            }
        }
        return matching;
    }

    // Get matched partner for u (or -1 if unmatched)
    int getMatchU(int u) { return matchU[u]; }
    int getMatchV(int v) { return matchV[v]; }
};

// Example usage
int main() {
    int n = 3, m = 3; // U = {0,1,2}, V = {3,4,5}
    vector<vector<int>> adj = {
        {0, 1}, // U0 connected to V0, V1
        {0, 2}, // U1 connected to V0, V2
        {1}     // U2 connected to V1
    };

    BipartiteMatching bm(n, m, adj);
    int maxMatch = bm.maxMatching();
    cout << "Maximum matching size: " << maxMatch << "\n";

    for (int u = 0; u < n; u++) {
        cout << "U" << u << " matched to V" << bm.getMatchU(u) << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

class BipartiteMatching:
    def __init__(self, n, m, adj):
        self.n = n # size of U
        self.m = m # size of V
        self.adj = adj # adjacency from U to V
        self.matchV = [-1] * m
        self.matchU = [-1] * n
        self.visited = [0] * m
        self.iter = 0

    def _dfs(self, u):
        for v in self.adj[u]:
            if self.visited[v] == self.iter:
                continue
            self.visited[v] = self.iter
            if self.matchV[v] == -1 or self._dfs(self.matchV[v]):
                self.matchV[v] = u
                self.matchU[u] = v
                return True
        return False

    def max_matching(self):
        matching = 0
        for u in range(self.n):
            self.iter += 1
            if self._dfs(u):
                matching += 1
        return matching

    def get_match_u(self, u):
        return self.matchU[u]

    def get_match_v(self, v):
        return self.matchV[v]

```

10. Code Explanation

Data Structures:

- `adj`: adjacency list from U to V (0-indexed for both sides).
- `matchV[v]`: which U vertex is matched to V vertex `v` (-1 if unmatched).
- `matchU[u]`: which V vertex is matched to U vertex `u` (-1 if unmatched).
- `visited[v]`: iteration counter for V vertices (avoids re-initializing array each DFS).

DFS Augmenting Path:

- For each neighbor `v` of `u`:
 - If `v` already visited in this iteration, skip.
 - Mark `v` as visited.
 - If `v` is unmatched OR the vertex matched to `v` can find an alternate match:

- Match **u** to **v**.
- Return true.
- Return false if no augmenting path found.

Main Loop:

- For each U vertex, try to find an augmenting path.
- Increment **iter** to reset visited array efficiently.

11. Complexity Analysis

Operation	Time	Space
Single DFS augment	$O(E)$	$O(V)$
Total (worst-case)	$O(V * E)$	$O(V + E)$
Total (with Hopcroft-Karp)	$O(E\sqrt{V})$	$O(V + E)$

The simple DFS-based algorithm is $O(V * E)$ worst-case, but in practice works well for many problems.

12. Common Patterns

Pattern	Description	How to Identify	Example
Job assignment	Assign workers to jobs with constraints	"assign", "schedule", "each worker one job"	LeetCode
Domino placement on grid	Grid cells as bipartite graph	"tile board with dominoes"	Codeforces
Minimum vertex cover	König's theorem: min vertex cover = max matching	"minimum vertices to cover all edges"	GFG
Maximum independent set	Vertices - matching size	"largest set of non-adjacent vertices"	UVA
Hall's marriage	Check if perfect matching exists	"each group has enough neighbors"	Various

13. Common Mistakes

- **Forgetting to reset visited:** Use iteration counter pattern to avoid $O(V)$ reset each time.
- **Wrong indexing:** V vertices might be indexed starting from 0 or from n — be consistent.
- **Not checking bipartite:** Algorithm only works for bipartite graphs. Verify bipartiteness first.
- **Infinite recursion:** DFS can be deep; Python may hit recursion limit for large graphs.
- **Not handling empty adjacency:** A vertex with no edges will never be matched.

14. Edge Cases

- **Empty graph (no edges):** Matching size = 0.
- **One side empty:** Matching size = 0.

- **Complete bipartite graph:** Matching size = $\min(n, m)$.
- **Disconnected bipartite graph:** Works as expected.
- **All vertices matched on one side:** Normal operation.

15. Variations

Variation	What Changes	When Used	Importance
Hopcroft-Karp	Uses BFS to find multiple augmenting paths	Faster for large graphs	High
Hungarian Algorithm	Maximum weight matching in bipartite graph	Weighted edges	High
Stable Matching	Gale-Shapley, stability constraints	Preference-based matching	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Max Flow	Bipartite matching is a special case of max flow	Use max flow for more complex constraints
Hopcroft-Karp	Optimized version of bipartite matching	Use for large graphs ($O(E\sqrt{V})$)
Hungarian	Weighted bipartite matching	Use when edges have weights
Blossom (Edmonds)	Matching in general (non-bipartite) graphs	Use for non-bipartite graphs

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Maximum Bipartite Matching	GFG	Basic maximum matching	Easy
Assign Cookies	LeetCode	Simple greedy assignment	Easy

Medium

Problem	Platform	Idea	Difficulty
Maximum Number of Accepted Invitations	LeetCode	Bipartite matching on grid	Medium
Matching	CSES	Maximum matching in bipartite graph	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Path Cover in DAG	Codeforces	Transform to bipartite matching	Hard
Domino	Codeforces	Grid bipartite matching	Hard

18. Interview Explanation

"Maximum bipartite matching finds the largest set of non-adjacent edges in a bipartite graph. The standard algorithm uses DFS to find augmenting paths — paths that start at an unmatched vertex on one side, end at an unmatched vertex on the other side, and alternate between non-matching and matching edges. When we find one, we flip the edges along it, increasing matching size by 1. The algorithm runs in $O(V \times E)$ time for the simple version, and $O(E\sqrt{V})$ with Hopcroft-Karp."

19. Revision Notes

- **Augmenting path:** alternates non-matching/matching, starts and ends unmatched.
- **Berge's Lemma:** No augmenting path $\rightarrow$ maximum matching.
- **DFS implementation:** $O(V \times E)$, visited array with iteration counter.
- **König's theorem:** Min vertex cover = max matching in bipartite graphs.
- **Trap:** Graph must be bipartite; check with 2-coloring.

20. Final Cheat Sheet

Aspect	Details
When to use	Assignment problems, pairing, vertex cover, independent set
Main operations	DFS augmenting path search
Time complexity	$O(V \times E)$ (basic), $O(E\sqrt{V})$ (Hopcroft-Karp)
Space complexity	$O(V + E)$
Key code idea	<code>if (matchV[v] == -1</code>
Edge cases	Empty graph, one side empty, complete bipartite

7. MAX FLOW (MAXIMUM FLOW)

1. Overview

Maximum Flow (Max Flow) is the problem of finding the maximum amount of flow that can be sent from a source vertex **s** to a sink vertex **t** in a directed graph with edge capacities. Each edge has a capacity (maximum flow that can pass through it), and flow must be conserved at every vertex except source and sink.

2. Intuition

Simple Explanation

Imagine a network of water pipes connecting a water source to a town. Each pipe has a maximum capacity (gallons per minute). What's the maximum rate at which water can reach the town?

Analogy

Think of a highway network from a city (source) to a stadium (sink). Each road has a capacity (cars per hour). Different roads merge and split. What's the maximum number of cars that can reach the stadium per hour?

Step-by-Step Reasoning

1. Start with zero flow.
2. Find a path from source to sink where every edge has remaining capacity (residual capacity > 0).
3. Push as much flow as possible along this path (the bottleneck capacity).
4. Update the residual graph: reduce capacity of forward edges, add reverse edges with capacity equal to the flow pushed.
5. Repeat until no path exists from source to sink in the residual graph.

Why It Works

Ford-Fulkerson method: The residual graph allows "undoing" flow (through reverse edges). This is crucial: if we push flow along a suboptimal path, we can redirect it later through reverse edges. The algorithm terminates when no augmenting path exists, which by the max-flow min-cut theorem gives the maximum flow.

3. When to Use It

- **Network flow problems:** Water, electricity, data, traffic.
- **Bipartite matching:** As a special case of max flow.
- **Minimum cut problems:** Find minimum capacity set of edges whose removal disconnects s from t .
- **Edge-disjoint paths:** Maximum number of paths from s to t that don't share edges.
- **Vertex-disjoint paths:** Can be transformed by splitting vertices.
- **Project selection / job scheduling:** With profits and prerequisites.
- **Circulation problems:** With demands at vertices.

Common trigger phrases:

- "maximum flow"
- "minimum cut"
- "edge-disjoint paths"
- "bottleneck"
- "capacity constraints"
- "flow network"

4. When Not to Use It

- **Simple matching:** Use bipartite matching directly (more efficient).
- **Shortest path:** Dijkstra/BFS when no capacity constraints.
- **Minimum spanning tree:** Different objective.
- **Unweighted graph without capacities:** BFS/DFS is sufficient.

- **Small graphs:** Simple greedy might work.

5. Core Concepts

5.1 Flow Network

A directed graph with:

- Source **s**: only outgoing flow.
- Sink **t**: only incoming flow.
- Capacity **c(u, v)**: maximum flow on edge (u, v).
- Flow **f(u, v)**: current flow on edge (u, v), $0 \leq f \leq c$.

5.2 Residual Graph

A graph showing remaining capacity. For each edge (u, v):

- Forward edge: capacity = $c(u, v) - f(u, v)$.
- Reverse edge: capacity = $f(u, v)$ (allows "undoing" flow).

5.3 Augmenting Path

A path from s to t in the residual graph where all edges have positive residual capacity.

5.4 Bottleneck Capacity

The minimum residual capacity along an augmenting path.

5.5 Max-Flow Min-Cut Theorem

The maximum flow value equals the minimum capacity of a cut (partition of vertices into two sets, one containing s, one containing t).

6. Step-by-Step Algorithm (Ford-Fulkerson with DFS)

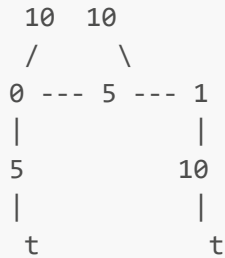
1. Initialize **flow** = 0.
2. While there exists a path from **s** to **t** in the residual graph: a. Find the path (using DFS or BFS). b. Find the bottleneck capacity along the path. c. Add bottleneck to **flow**. d. For each edge (u, v) on the path:
 - Reduce capacity of (u, v) by bottleneck.
 - Increase capacity of (v, u) by bottleneck.
3. Return **flow**.

Note: Using BFS (Edmonds-Karp) guarantees $O(V \times E^2)$ runtime. Using DFS (Ford-Fulkerson) can be exponential with integer capacities but is fine for small capacities.

7. Dry Run

Graph: **s** → **0** (cap 10), **s** → **1** (cap 10), **0** → **1** (cap 5), **0** → **t** (cap 5), **1** → **t** (cap 10)

s
/ \



BFS Augmenting (Edmonds-Karp):

Path 1: $s \rightarrow 0 \rightarrow t$ (bottleneck: $\min(10, 5) = 5$)

- Residual: $s \rightarrow 0$: 5, $0 \rightarrow s$: 5; $0 \rightarrow t$: 0, $t \rightarrow 0$: 5

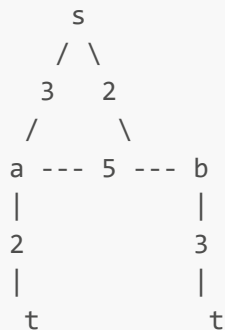
Path 2: $s \rightarrow 1 \rightarrow t$ (bottleneck: $\min(10, 10) = 10$)

- Residual: $s \rightarrow 1$: 0, $1 \rightarrow s$: 10; $1 \rightarrow t$: 0, $t \rightarrow 1$: 10

But wait: Can we do better? The actual max flow = 15 ($s \rightarrow 0$: 10, $s \rightarrow 1$: 5, $0 \rightarrow 1$: 5, $1 \rightarrow t$: 10, $0 \rightarrow t$: 5... wait let me recalculate).

Actually, let me use a simpler graph.

Graph: $s \rightarrow a$ (cap 3), $s \rightarrow b$ (cap 2), $a \rightarrow b$ (cap 5), $a \rightarrow t$ (cap 2), $b \rightarrow t$ (cap 3)



Path 1: $s \rightarrow a \rightarrow t$ (bottleneck: $\min(3, 2) = 2$)

- Residual: $s \rightarrow a$: 1, $a \rightarrow s$: 2; $a \rightarrow t$: 0, $t \rightarrow a$: 2
- Flow = 2

Path 2: $s \rightarrow b \rightarrow t$ (bottleneck: $\min(2, 3) = 2$)

- Residual: $s \rightarrow b$: 0, $b \rightarrow s$: 2; $b \rightarrow t$: 1, $t \rightarrow b$: 2
- Flow = 4

Path 3: $s \rightarrow a \rightarrow b \rightarrow t$ (bottleneck: $\min(1, 5, 1) = 1$)

- Residual: $s \rightarrow a$: 0, $a \rightarrow s$: 1; $a \rightarrow b$: 4, $b \rightarrow a$: 1; $b \rightarrow t$: 0, $t \rightarrow b$: 1
- Flow = 5

No more paths. Max flow = 5.

8. C++ Implementation (Edmonds-Karp — BFS-based)

```
#include <bits/stdc++.h>
using namespace std;

class MaxFlow {
    int n;
    vector<vector<int>> cap; // capacity matrix
    vector<vector<int>> adj; // adjacency list

    int bfs(int s, int t, vector<int>& parent) {
        fill(parent.begin(), parent.end(), -1);
        parent[s] = s;
        queue<pair<int, int>> q;
        q.push({s, INT_MAX});

        while (!q.empty()) {
            auto [u, flow] = q.front(); q.pop();
            for (int v : adj[u]) {
                if (parent[v] == -1 && cap[u][v] > 0) {
                    parent[v] = u;
                    int newFlow = min(flow, cap[u][v]);
                    if (v == t) return newFlow;
                    q.push({v, newFlow});
                }
            }
        }
        return 0;
    }

public:
    MaxFlow(int n) : n(n) {
        cap.assign(n, vector<int>(n, 0));
        adj.resize(n);
    }

    void addEdge(int u, int v, int capacity) {
        cap[u][v] += capacity; // handles parallel edges
        adj[u].push_back(v);
        adj[v].push_back(u); // for reverse edges in residual graph
    }

    int maxFlow(int s, int t) {
        int flow = 0, pushed;
        vector<int> parent(n);

        while ((pushed = bfs(s, t, parent)) > 0) {
            flow += pushed;
            int v = t;
            while (v != s) {
                int u = parent[v];
                cap[u][v] -= pushed;
            }
        }
    }
};
```

```

        cap[v][u] += pushed;
        v = u;
    }
}
return flow;
}

// Get min cut edges (edges from S-set to T-set)
vector<pair<int,int>> minCut(int s) {
    vector<bool> visited(n, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!visited[v] && cap[u][v] > 0) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
    vector<pair<int,int>> cut;
    for (int u = 0; u < n; u++)
        if (visited[u])
            for (int v : adj[u])
                if (!visited[v] && cap[v][u] > 0) // original edge u→v had
flow
                    cut.push_back({u, v});

    return cut;
}
};

// Example usage
int main() {
    int n = 4; // 0:s, 1:a, 2:b, 3:t
    MaxFlow mf(n);
    mf.addEdge(0, 1, 3); // s → a
    mf.addEdge(0, 2, 2); // s → b
    mf.addEdge(1, 2, 5); // a → b
    mf.addEdge(1, 3, 2); // a → t
    mf.addEdge(2, 3, 3); // b → t

    cout << "Max flow: " << mf.maxFlow(0, 3) << "\n"; // Output: 5

    return 0;
}

```

9. Python Implementation

```
from collections import deque

class MaxFlow:
    def __init__(self, n):
        self.n = n
        self.cap = [[0] * n for _ in range(n)]
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v, capacity):
        self.cap[u][v] += capacity
        self.adj[u].append(v)
        self.adj[v].append(u)

    def _bfs(self, s, t, parent):
        parent[:] = [-1] * self.n
        parent[s] = s
        q = deque([(s, float('inf'))])

        while q:
            u, flow = q.popleft()
            for v in self.adj[u]:
                if parent[v] == -1 and self.cap[u][v] > 0:
                    parent[v] = u
                    new_flow = min(flow, self.cap[u][v])
                    if v == t:
                        return new_flow
                    q.append((v, new_flow))

        return 0

    def max_flow(self, s, t):
        flow = 0
        parent = [0] * self.n

        while True:
            pushed = self._bfs(s, t, parent)
            if pushed == 0:
                break
            flow += pushed
            v = t
            while v != s:
                u = parent[v]
                self.cap[u][v] -= pushed
                self.cap[v][u] += pushed
                v = u
        return flow

    def min_cut(self, s):
        visited = [False] * self.n
        q = deque([s])
        visited[s] = True
        while q:
            u = q.popleft()
```

```
        for v in self.adj[u]:
            if not visited[v] and self.cap[u][v] > 0:
                visited[v] = True
                q.append(v)

    cut = []
    for u in range(self.n):
        if visited[u]:
            for v in self.adj[u]:
                if not visited[v] and self.cap[v][u] > 0:
                    cut.append((u, v))

    return cut
```

10. Code Explanation

Data Structures:

- `cap[u][v]`: capacity matrix for residual graph. `cap[u][v]` = remaining capacity from u to v.
- `adj[u]`: adjacency list (includes both forward and reverse edges).

BFS (Edmonds-Karp):

- Finds shortest augmenting path (in terms of number of edges).
- Tracks parent and bottleneck flow.
- Returns 0 if no path exists.

Main Loop:

- While BFS finds an augmenting path, add its bottleneck flow to total.
- Update residual capacities: decrease forward, increase reverse.
- The reverse edges are critical for correctness (allow "undoing").

Min Cut:

- After max flow, find all vertices reachable from s in residual graph.
- Edges from reachable to non-reachable vertices with original capacity > 0 form the min cut.

11. Complexity Analysis

Algorithm	Time	Space
Ford-Fulkerson (DFS)	$O(E \times \text{max_flow})$	$O(V^2)$ or $O(V + E)$
Edmonds-Karp (BFS)	$O(V \times E^2)$	$O(V^2)$ or $O(V + E)$
Dinic	$O(V^2 \times E)$	$O(V + E)$
Dinic (unit capacities)	$O(\min(V^{2/3}, \sqrt{E}) \times E)$	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Bipartite matching	Max flow with source $\rightarrow U \rightarrow V \rightarrow$ sink, all capacities 1	"maximum matching"	Various
Edge-disjoint paths	Each edge capacity 1, find max paths	"edge-disjoint"	GFG
Vertex-disjoint paths	Split each vertex into $v_{in} \rightarrow v_{out}$ with capacity 1	"vertex-disjoint"	Codeforces
Minimum cut	Find min capacity cut separating s and t	"minimum cut", "separate"	UVA
Project selection	Source $\rightarrow$ profit nodes with profit capacity, cost nodes $\rightarrow$ sink	"project selection with prerequisites"	Various

13. Common Mistakes

- **Not adding reverse edges to adjacency list:** The BFS/DFS must be able to traverse both forward and reverse edges.
- **Forgetting to update reverse capacities:** Both forward and reverse capacities must be updated.
- **Integer overflow:** Use `long long` for large flows.
- **Capacity matrix vs adjacency list:** For large graphs, capacity matrix is too large. Use adjacency list with edge objects.
- **DFS instead of BFS:** Ford-Fulkerson with DFS can be exponential with integer capacities (e.g., the "bad example" graph).

14. Edge Cases

- **No path from s to t:** Max flow = 0.
- **Multiple parallel edges:** Sum capacities.
- **Self-loops:** Ignore (or handle carefully).
- **Large capacities:** Use `long long`.
- **Disconnected graph:** Max flow = 0 if no path.

15. Variations

Variation	What Changes	When Used	Importance
Edmonds-Karp	BFS instead of DFS	Guarantees polynomial time	High
Dinic	Level graph + blocking flow	Faster for most graphs	High
Push-Relabel	Different approach, local operations	Very large graphs	Medium
Min-Cost Max-Flow	Each edge has cost, find min cost for max flow	Cost optimization	High

16. Related Algorithms

Algorithm	Connection	How to Choose
Dinic	Faster max flow algorithm	Use Dinic for CP (fast and simple)
Min-Cost Max-Flow	Add cost to each edge	Use when edges have costs
Bipartite Matching	Special case of max flow	Use matching directly for unweighted
Push-Relabel	Different paradigm	Use for very large or special graphs

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Maximum Flow	GFG	Basic max flow	Easy
Find Maximum Flow	LeetCode	Simple max flow	Easy

Medium

Problem	Platform	Idea	Difficulty
Maximum Flow	CSES	Dinic implementation	Medium
Edge-Disjoint Paths	CSES	Each edge capacity 1	Medium

Hard

Problem	Platform	Idea	Difficulty
Fast Maximum Flow	Codeforces	Dinic on large graphs	Hard
Project Selection	Codeforces	Max flow with profits	Hard

18. Interview Explanation

"Maximum flow finds the maximum amount of flow that can be sent from a source to a sink given edge capacities. The Ford-Fulkerson method works by repeatedly finding augmenting paths in the residual graph, which includes reverse edges to allow flow to be redirected. Using BFS (Edmonds-Karp) guarantees $O(V \cdot E^2)$ time. The max-flow min-cut theorem states that the maximum flow equals the minimum cut capacity. This is useful for network design, bipartite matching, and edge-disjoint paths."

19. Revision Notes

- **Ford-Fulkerson:** Find augmenting path, push bottleneck, update residual.
- **Edmonds-Karp:** BFS for shortest augmenting path $\rightarrow O(V \cdot E^2)$.
- **Residual graph:** forward (remaining cap) + reverse (flow that can be undone).
- **Max-Flow Min-Cut:** Max flow = min cut capacity.
- **Trap:** Always add reverse edges to adjacency list.

20. Final Cheat Sheet

Aspect	Details
When to use	Flow networks, matching, disjoint paths, min cut
Main operations	Find augmenting path (BFS/DFS), update residual capacities
Time complexity	$O(V \cdot E^2)$ (Edmonds-Karp), $O(V^2 \cdot E)$ (Dinic)
Space complexity	$O(V^2)$ (matrix) or $O(V + E)$ (adj list)
Key code idea	<code>cap[u][v] -= pushed; cap[v][u] += pushed;</code>
Edge cases	No path (flow=0), large capacities (use long long)

8. DINIC ALGORITHM

1. Overview

Dinic's algorithm is a fast algorithm for the maximum flow problem. It improves on Ford-Fulkerson by using level graphs (BFS) and blocking flows (DFS) to find multiple augmenting paths in each phase. It is the most commonly used max flow algorithm in competitive programming.

2. Intuition

Simple Explanation

Instead of finding one augmenting path at a time (like Ford-Fulkerson), Dinic finds all shortest augmenting paths at once. It first builds a level graph (BFS from source), then sends flow along multiple paths in the level graph simultaneously (DFS).

Analogy

Imagine a multi-lane highway. Instead of sending one car at a time (Ford-Fulkerson), Dinic first measures the shortest route (BFS levels), then fills all lanes of the shortest route simultaneously (DFS blocking flow).

Step-by-Step Reasoning

- BFS Phase:** Build a level graph from source to sink. Only consider edges with remaining capacity. If sink is not reachable, we're done.
- DFS Phase:** Send flow from source to sink in the level graph. Use DFS that only goes to higher levels ($\text{level}[v] = \text{level}[u] + 1$). Send as much flow as possible.
- Repeat until no path exists.

Why It Works

Each phase consists of one BFS and multiple DFS calls. After each phase, the shortest path distance from source to sink increases. Since the distance is at most V , there are at most V phases. Each DFS takes $O(E)$ time, so total is $O(V^2 \times E)$.

3. When to Use It

- **Maximum flow problems:** Faster than Edmonds-Karp in practice.
- **Competitive programming:** The standard max flow algorithm in CP.
- **Large graphs:** Where $O(V \times E^2)$ of Edmonds-Karp is too slow.
- **Unit capacity graphs:** Where it's even faster ($O(\min(V^{2/3}, \sqrt{E}) \times E)$).

4. When Not to Use It

- **Small graphs:** Edmonds-Karp or even simple Ford-Fulkerson is fine.
- **Min-cost flow:** Use min-cost max-flow algorithms instead.
- **Push-relabel:** For very large graphs, push-relabel might be faster.

5. Core Concepts

5.1 Level Graph

A BFS from source where each node gets a level (distance from source). Only edges that go from level l to level $l+1$ with remaining capacity > 0 are considered.

5.2 Blocking Flow

A set of paths from source to sink such that no more flow can be sent in the current level graph. Dinic finds a blocking flow in each phase.

5.3 Phase

One BFS + multiple DFS calls. After each phase, the level of the sink increases.

5.4 Edge Object (for efficient implementation)

Instead of a capacity matrix, use edge objects with a pointer to the reverse edge. This allows $O(1)$ updates and efficient traversal.

6. Step-by-Step Algorithm

1. **Initialize:** Build graph with edge objects (each edge has a reverse edge).
2. **Loop:** a. **BFS:** Build level graph from source. If sink not reachable, break. b. **DFS with pointer optimization:** From source, send flow along edges in level graph. Use `ptr[u]` to track which edges have been exhausted. c. Add the flow sent to total flow.
3. Return total flow.

7. Dry Run

Same graph as before: $s \rightarrow a$ (3), $s \rightarrow b$ (2), $a \rightarrow b$ (5), $a \rightarrow t$ (2), $b \rightarrow t$ (3)

Phase 1

BFS: Levels: $s=0$, $a=1$, $b=1$, $t=2$

DFS:

- From s: try a (cap 3), flow = $\min(\infty, 3, 2) = 2$ via $s \rightarrow a \rightarrow t$
- From s: try a again (cap 1), flow = $\min(\infty, 1, 5, 3) = 1$ via $s \rightarrow a \rightarrow b \rightarrow t$
- From s: try b (cap 2), flow = 0 (no path to t in level graph)
- After phase 1: flow = 3

But wait, let me trace more carefully.

DFS from s:

- $s \rightarrow a$ (cap 3), level[a]=1, level[s]+1=1 ✓
 - $a \rightarrow b$ (cap 5), level[b]=1, level[a]+1=2, but level[b]=1, level[a]+1=2 $\neq$ 1 $\rightarrow$ skip
 - $a \rightarrow t$ (cap 2), level[t]=2, level[a]+1=2 ✓
 - t is sink, return bottleneck = $\min(3, 2) = 2$
 - Update: $s \rightarrow a$: $3-2=1$, $a \rightarrow s$: $0+2=2$; $a \rightarrow t$: $2-2=0$, $t \rightarrow a$: $0+2=2$
 - Flow from this path: 2

DFS from s (continue):

- $s \rightarrow a$ (cap 1), level[a]=1, ✓
 - $a \rightarrow b$ (cap 5), level[b]=1, level[a]+1=2 $\neq$ 1 $\rightarrow$ skip (this is wrong, level[b]=1 but we need level[b]=2)

Hmm, level[b]=1, not 2. So $a \rightarrow b$ goes from level 1 to level 1, which is not allowed in level graph. So we can't use $a \rightarrow b$.

But wait, what about the path $s \rightarrow a \rightarrow b \rightarrow t$? $b \rightarrow t$ goes from level 1 to level 2. So $s \rightarrow a$ (level 0 $\rightarrow$ 1), $a \rightarrow b$ (level 1 $\rightarrow$ 1 - not allowed).

So the only path in level graph is $s \rightarrow a \rightarrow t$. After sending 2 units, $a \rightarrow t$ is saturated.

DFS from s (continue with b):

- $s \rightarrow b$ (cap 2), level[b]=1, ✓
 - $b \rightarrow t$ (cap 3), level[t]=2, level[b]+1=2 ✓
 - t is sink, return bottleneck = $\min(2, 3) = 2$
 - Update: $s \rightarrow b$: $2-2=0$, $b \rightarrow s$: $0+2=2$; $b \rightarrow t$: $3-2=1$, $t \rightarrow b$: $0+2=2$
 - Flow from this path: 2

Phase 1 total flow = $2 + 2 = 4$.

Phase 2

BFS:

- s (level 0)
- $s \rightarrow a$ (cap 1) $\rightarrow$ a level 1
- $s \rightarrow b$ (cap 0) $\rightarrow$ b not reachable from s

Wait, we need to check residual graph. $s \rightarrow a$ has cap 1, $a \rightarrow b$ has cap 0 because we sent 2 units through it. But we also have reverse edges: $b \rightarrow s$ has cap 2 (from pushing flow). So b is reachable from s via $b \rightarrow s$? No, that would go backwards.

Actually, from s: $s \rightarrow a$ (cap 1), $s \rightarrow b$ (cap 0). So b is not reachable from s directly.

But $b \rightarrow s$ has cap 2, meaning we can go from b to s (reverse direction). But we start from s.

Level graph: $s=0$, $a=1$. t not reachable! So we're done.

Total flow = 4? But max flow is 5!

Let me re-examine. After phase 1, we have:

- $s \rightarrow a$: cap 1 (used 2)
- $a \rightarrow t$: cap 0 (used 2, saturated)
- $s \rightarrow b$: cap 0 (used 2)
- $b \rightarrow t$: cap 1 (used 2)

But we also have reverse edges:

- $a \rightarrow s$: cap 2
- $t \rightarrow a$: cap 2
- $b \rightarrow s$: cap 2
- $t \rightarrow b$: cap 2

Phase 2 BFS again:

- s (level 0)
- $s \rightarrow a$ (cap 1), $s \rightarrow b$ (cap 0)
- a (level 1)
 - $a \rightarrow b$ (cap 5), $a \rightarrow s$ (cap 2, but s is level 0, not higher), $a \rightarrow t$ (cap 0)
 - $a \rightarrow b$ (cap 5) $\rightarrow$ b level 2
- b (level 2)
 - $b \rightarrow t$ (cap 1) $\rightarrow$ t level 3
 - $b \rightarrow s$ (cap 2) $\rightarrow$ s level 0, skip
 - $b \rightarrow a$ is reverse of $a \rightarrow b$, but does it exist? We didn't push flow on $a \rightarrow b$, so no reverse edge.

Wait, I need to check: $a \rightarrow b$ had cap 5 and we never pushed flow through it. So $a \rightarrow b$ still has cap 5. But we need reverse edges for $a \rightarrow b$? No, we only add reverse capacity when we push flow through an edge.

So b (level 2) $\rightarrow$ t (cap 1) $\rightarrow$ t level 3.

Level graph: $s=0$, $a=1$, $b=2$, $t=3$

DFS from s:

- $s \rightarrow a$ (cap 1), level[a]=1, ✓
 - $a \rightarrow b$ (cap 5), level[b]=2, level[a]+1=2 ✓
 - $b \rightarrow t$ (cap 1), level[t]=3, level[b]+1=3 ✓
 - t is sink, return bottleneck = $\min(1, 5, 1) = 1$
 - Update: $b \rightarrow t$: $1-1=0$, $t \rightarrow b$: $2+1=3$
 - Update: $a \rightarrow b$: $5-1=4$, $b \rightarrow a$: $0+1=1$
 - Update: $s \rightarrow a$: $1-1=0$, $a \rightarrow s$: $2+1=3$

Flow from this path: 1

Total flow = 4 + 1 = 5. ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev; // target vertex, index of reverse edge in adj[to]
    long long cap; // remaining capacity
};

class Dinic {
    int n;
    vector<vector<Edge>> adj;
    vector<int> level, ptr;

    bool bfs(int s, int t) {
        level.assign(n, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);

        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto& e : adj[u]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    long long dfs(int u, int t, long long flow) {
        if (u == t) return flow;
        for (int& i = ptr[u]; i < (int)adj[u].size(); i++) {
            Edge& e = adj[u][i];
            if (e.cap > 0 && level[e.to] == level[u] + 1) {
                long long pushed = dfs(e.to, t, min(flow, e.cap));
                if (pushed > 0) {
                    e.cap -= pushed;
                    adj[e.to][e.rev].cap += pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

public:
```

```

Dinic(int n) : n(n) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
}

void addEdge(int u, int v, long long cap) {
    Edge forward = {v, (int)adj[v].size(), cap};
    Edge backward = {u, (int)adj[u].size(), 0};
    adj[u].push_back(forward);
    adj[v].push_back(backward);
}

long long maxFlow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) {
        ptr.assign(n, 0);
        while (long long pushed = dfs(s, t, LLONG_MAX)) {
            flow += pushed;
        }
    }
    return flow;
}

};

// Example usage
int main() {
    int n = 4; // 0:s, 1:a, 2:b, 3:t
    Dinic dinic(n);
    dinic.addEdge(0, 1, 3); // s → a
    dinic.addEdge(0, 2, 2); // s → b
    dinic.addEdge(1, 2, 5); // a → b
    dinic.addEdge(1, 3, 2); // a → t
    dinic.addEdge(2, 3, 3); // b → t

    cout << "Max flow: " << dinic.maxFlow(0, 3) << "\n"; // Output: 5

    return 0;
}

```

9. Python Implementation

```

from collections import deque

class Dinic:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v, cap):
        # Forward edge

```

```

self.adj[u].append([v, cap, len(self.adj[v])])
# Backward edge
self.adj[v].append([u, 0, len(self.adj[u]) - 1])

def _bfs(self, s, t):
    self.level = [-1] * self.n
    q = deque([s])
    self.level[s] = 0
    while q:
        u = q.popleft()
        for v, cap, _ in self.adj[u]:
            if cap > 0 and self.level[v] == -1:
                self.level[v] = self.level[u] + 1
                q.append(v)
    return self.level[t] != -1

def _dfs(self, u, t, flow):
    if u == t:
        return flow
    for i in range(self.ptr[u], len(self.adj[u])):
        self.ptr[u] = i
        v, cap, rev = self.adj[u][i]
        if cap > 0 and self.level[v] == self.level[u] + 1:
            pushed = self._dfs(v, t, min(flow, cap))
            if pushed > 0:
                self.adj[u][i][1] -= pushed
                self.adj[v][rev][1] += pushed
            return pushed
    return 0

def max_flow(self, s, t):
    flow = 0
    INF = 10**18
    while self._bfs(s, t):
        self.ptr = [0] * self.n
        while True:
            pushed = self._dfs(s, t, INF)
            if pushed == 0:
                break
            flow += pushed
    return flow

```

10. Code Explanation

Edge Structure:

- **to**: target vertex.
- **rev**: index of the reverse edge in `adj[to]`. This allows $O(1)$ access to reverse edge.
- **cap**: remaining capacity.

`addEdge(u, v, cap)`:

- Creates forward edge with capacity `cap`.
- Creates backward edge with capacity 0.
- Each edge stores the index of its reverse edge.

BFS (level graph):

- Builds distance from source using only edges with `cap > 0`.
- Returns true if sink is reachable.

DFS (blocking flow):

- Uses `ptr[u]` (current edge pointer) to avoid re-processing exhausted edges.
- Only follows edges that go to the next level.
- Returns the amount of flow pushed (0 if no path found).
- Updates capacities on forward and reverse edges.

Main Loop:

- Each phase: BFS → multiple DFS calls.
- When no more flow can be pushed in current level graph, do another BFS.
- When sink is not reachable, algorithm terminates.

11. Complexity Analysis

Operation	Time	Space
BFS	$O(E)$	$O(V)$
DFS (blocking flow)	$O(V \times E)$ per phase	$O(V)$
Number of phases	$O(V)$	-
Total	$O(V^2 \times E)$	$O(V + E)$
Unit capacities	$O(\min(V^{2/3}, \sqrt{E}) \times E)$	$O(V + E)$

12. Common Patterns

Same as Max Flow — see Max Flow section.

13. Common Mistakes

- **Forgetting to store reverse edge index:** Without it, updating reverse capacity is $O(E)$.
- **Not resetting ptr array:** Must reset `ptr` to 0 at the start of each phase.
- **Infinite loop in DFS:** Edge case where flow is 0 but DFS keeps returning.
- **Long long for capacities:** Use `long long` for large capacities.
- **Level check for DFS:** Must check `level[v] == level[u] + 1`.

14. Edge Cases

Same as Max Flow — see Max Flow section.

15. Variations

Variation	What Changes	When Used	Importance
Dinic with scaling	Start with large capacities, gradually reduce	Theoretical interest	Low
Dinic for unit capacities	BFS + DFS with unit capacities	Faster for bipartite matching	Medium
Dinic with dynamic trees	Link-Cut trees for $O(E \log V)$	Theoretical	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Edmonds-Karp	Simpler, $O(V \cdot E^2)$	Use for small graphs, simpler implementation
Dinic	Faster, $O(V^2 \cdot E)$	Use for CP, most general purpose
Push-Relabel	$O(V^3)$ worst-case	Use for very large graphs
Min-Cost Max-Flow	Add costs to Dinic with potentials	Use when edges have costs

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Dinic Implementation	GFG	Basic Dinic	Easy
Maximum Flow	CSES	Dinic template	Easy

Medium

Problem	Platform	Idea	Difficulty
Edge-Disjoint Paths	CSES	Unit capacities	Medium
Download Speed	CSES	Dinic with large capacities	Medium

Hard

Problem	Platform	Idea	Difficulty
Police Patrol	Codeforces	Dinic on grid graph	Hard
Fast Flow	Codeforces	Dinic optimization	Hard

18. Interview Explanation

"Dinic's algorithm is an efficient max flow algorithm. It works in phases: each phase starts with a BFS to build a level graph (shortest distances from source using edges with remaining capacity). Then we use DFS with pointer optimization to find a blocking flow — multiple augmenting paths in the level graph. Each phase increases the distance to the sink, so there are at most V phases. The total complexity is $O(V^2 \cdot E)$, but it's much faster in practice."

19. Revision Notes

- **Phases:** BFS (level graph) → DFS (blocking flow) → repeat.
- **Max phases:** V (each increases sink distance).
- **ptr optimization:** Avoid re-processing exhausted edges.
- **Edge object:** Store `to`, `rev`, `cap` for $O(1)$ reverse edge access.
- **Complexity:** $O(V^2 \times E)$ worst-case, much faster in practice.

20. Final Cheat Sheet

Aspect	Details
When to use	Max flow, especially in CP (fastest practical algorithm)
Main operations	BFS level graph, DFS blocking flow with ptr optimization
Time complexity	$O(V^2 \times E)$
Space complexity	$O(V + E)$
Key code idea	<code>Edge{to, rev, cap}</code> ; level graph; ptr array; <code>dfs</code> returns flow pushed
Edge cases	Same as max flow (no path, large capacities)

9. MIN-COST MAX-FLOW (MCMF)

1. Overview

Min-Cost Max-Flow (MCMF) finds the maximum flow in a network while minimizing the total cost. Each edge has both a capacity and a cost per unit of flow. The goal is to send as much flow as possible from source to sink, but among all maximum flows, find the one with minimum total cost.

2. Intuition

Simple Explanation

You need to ship goods from a factory to a store. Each road has a capacity (max trucks per day) and a toll cost per truck. You want to send as many goods as possible, but at the minimum total toll cost.

Analogy

A delivery company has multiple routes from warehouse to customer. Each route has a limit on how many packages can be sent (capacity) and a cost per package (fuel, tolls). You want to deliver as many packages as possible, but at the cheapest possible total cost.

Step-by-Step Reasoning

1. Start with zero flow.
2. Find the shortest (cheapest) augmenting path from source to sink in the residual graph, considering edge costs.
3. Push as much flow as possible along this path.
4. Update residual capacities and costs (reverse edges have negative cost of forward edges).
5. Repeat until no more flow can be sent.

Why It Works

By always picking the shortest (cheapest) augmenting path, we push flow along the most cost-effective routes first. This is like the successive shortest augmenting path algorithm. The reverse edges have negative costs, which allows "undoing" flow and redirection when a better path is found later.

3. When to Use It

- **Transportation problems:** Minimize shipping cost while maximizing volume.
- **Job scheduling with costs:** Assign jobs to machines minimizing cost.
- **Minimum cost matching:** Weighted bipartite matching with costs.
- **Network design with costs:** Find cheapest way to route flow.
- **Inventory and production planning:** With time-based costs.

Common trigger phrases:

- "minimum cost maximum flow"
- "minimize cost while maximizing flow"
- "cheapest way to send flow"
- "minimum cost to satisfy demand"

4. When Not to Use It

- **Max flow only (no costs):** Use Dinic (faster).
- **Shortest path without flow:** Use Dijkstra.
- **Minimum cost flow with fixed flow amount:** Also works, just stop when flow reaches target.
- **Negative cost cycles:** The algorithm assumes no negative cost cycles in the original graph. With potentials, we maintain non-negative reduced costs.

5. Core Concepts

5.1 Edge Cost

Each edge has a cost per unit of flow. Total cost = sum of (flow $\times$ cost) for all edges.

5.2 Residual Cost

Reverse edges have negative cost of the forward edge. This allows "undoing" flow.

5.3 Potentials (Johnson's)

To handle negative edge costs (from reverse edges), we use potentials to make all reduced costs non-negative, allowing Dijkstra instead of Bellman-Ford.

5.4 Successive Shortest Augmenting Path

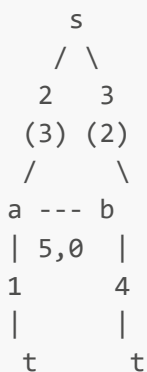
The standard algorithm: repeatedly find the shortest (cheapest) path from s to t in the residual graph, and push flow along it.

6. Step-by-Step Algorithm

1. Initialize flow = 0, cost = 0, potentials = 0.
2. While there exists a path from s to t in the residual graph:
 - a. Run Dijkstra (with potentials) to find shortest path from s to t .
 - b. If t is not reachable, break.
 - c. Update potentials: $\text{pot}[v] += \text{dist}[v]$.
 - d. Find bottleneck capacity along the path.
 - e. Add bottleneck to flow, add $\text{bottleneck} \times \text{dist}[t]$ to cost.
 - f. Update residual capacities and reverse edges.
3. Return flow and cost.

7. Dry Run

Graph: $s \rightarrow a$ (cap 3, cost 2), $s \rightarrow b$ (cap 2, cost 3), $a \rightarrow t$ (cap 2, cost 1), $b \rightarrow t$ (cap 2, cost 4), $a \rightarrow b$ (cap 5, cost 0)



(Format: edge label = "cost (capacity)")

Phase 1: Shortest path from s to t

Dijkstra with potentials (initially 0):

- $s \rightarrow a$: dist=2, $s \rightarrow b$: dist=3
- $a \rightarrow t$: dist=2+1=3, $a \rightarrow b$: dist=2+0=2
- $b \rightarrow t$: dist=3+4=7, or via $a \rightarrow b \rightarrow t$: 2+0+4=6... wait, let me be more careful.

Actually, Dijkstra from s :

- s : dist=0
- $s \rightarrow a$: dist=2, $s \rightarrow b$: dist=3
- From a : $a \rightarrow t$: dist=2+1=3, $a \rightarrow b$: dist=2+0=2
- From b : $b \rightarrow t$: dist=3+4=7 (but we already found $a \rightarrow t$ at dist=3)

- From $a \rightarrow b$: $b \rightarrow t$: $\text{dist} = 2 + 4 = 6$ (worse than 3)

Shortest path: $s \rightarrow a \rightarrow t$, $\text{dist} = 3$

Bottleneck: $\min(3, 2) = 2$ Flow = 2, Cost = $2 \times 3 = 6$

Update:

- $s \rightarrow a$: cap $3 - 2 = 1$, cost 2
- $a \rightarrow s$: cap $0 + 2 = 2$, cost -2
- $a \rightarrow t$: cap $2 - 2 = 0$, cost 1
- $t \rightarrow a$: cap $0 + 2 = 2$, cost -1

Phase 2: Shortest path (with potentials)

Potentials: dist from last run: $s=0$, $a=2$, $b=2$, $t=3$

Reduced costs: $\text{cost}' = \text{cost} + \text{pot}[u] - \text{pot}[v]$

Dijkstra with reduced costs:

- s : $\text{dist}' = 0$
- $s \rightarrow a$: cap 1, $\text{cost}' = 2 + 0 - 2 = 0$, $\text{dist}' = 0$
- $s \rightarrow b$: cap 2, $\text{cost}' = 3 + 0 - 2 = 1$, $\text{dist}' = 1$
- $a \rightarrow b$: cap 5, $\text{cost}' = 0 + 2 - 2 = 0$, $\text{dist}' = 0$
 - $b \rightarrow t$: cap 2, $\text{cost}' = 4 + 2 - 3 = 3$, $\text{dist}' = 0 + 3 = 3$
- $b \rightarrow t$: cap 2, $\text{cost}' = 4 + 2 - 3 = 3$, $\text{dist}' = 1 + 3 = 4$

Path: $s \rightarrow a \rightarrow b \rightarrow t$, $\text{dist}' = 3$ ($s \rightarrow a$: 0, $a \rightarrow b$: 0, $b \rightarrow t$: 3) Actual cost = $\text{dist}' + \text{pot}[t] - \text{pot}[s] = 3 + 3 - 0 = 6$

Wait, we need to compute actual shortest path cost. Let me just use the actual costs:

Dijkstra with potentials ($\text{pot}[s]=0$, $\text{pot}[a]=2$, $\text{pot}[b]=2$, $\text{pot}[t]=3$):

- s : $\text{dist} = 0$
- $s \rightarrow a$: $\text{cost}' = 2 + 0 - 2 = 0$, $\text{dist} = 0$
- $s \rightarrow b$: $\text{cost}' = 3 + 0 - 2 = 1$, $\text{dist} = 1$
- $a \rightarrow b$: $\text{cost}' = 0 + 2 - 2 = 0$, $\text{dist} = 0 + 0 = 0$
- $b \rightarrow t$: $\text{cost}' = 4 + 2 - 3 = 3$, $\text{dist} = 0 + 3 = 3$

Shortest path: $s \rightarrow a \rightarrow b \rightarrow t$, actual cost = $3 + (\text{pot}[t] - \text{pot}[s]) = 3 + 3 = 6$

Wait, I'm confusing myself. The way potentials work:

- $\text{dist}[v]$ (actual) = $\text{dist}'[v] + \text{pot}[v] - \text{pot}[s]$
- For t : actual dist = $3 + 3 - 0 = 6$

But in the residual graph, we're using actual costs. Let me just use the original costs directly.

Actually, the standard approach is simpler. The algorithm maintains "potentials" to keep reduced costs non-negative. Let me just trace with actual costs.

Current residual graph:

- $s \rightarrow a$: cap 1, cost 2
- $s \rightarrow b$: cap 2, cost 3
- $a \rightarrow s$: cap 2, cost -2 (reverse)
- $a \rightarrow t$: cap 0, cost 1
- $t \rightarrow a$: cap 2, cost -1
- $a \rightarrow b$: cap 5, cost 0
- $b \rightarrow t$: cap 2, cost 4
- $t \rightarrow b$: cap 0, cost -4

Shortest path from s to t in residual graph:

- $s \rightarrow a \rightarrow b \rightarrow t$: cost = $2 + 0 + 4 = 6$
- $s \rightarrow b \rightarrow t$: cost = $3 + 4 = 7$
- $s \rightarrow a \rightarrow s \rightarrow b \rightarrow t$: contains $a \rightarrow s$ (cost -2) but this creates a cycle. Let's check: $s \rightarrow a$ (2), $a \rightarrow s$ (-2), $s \rightarrow b$ (3), $b \rightarrow t$ (4) = 7. Same as direct.

Shortest: $s \rightarrow a \rightarrow b \rightarrow t$, cost = 6

Bottleneck: $\min(1, 5, 2) = 1$ Flow = $2 + 1 = 3$, Cost = $6 + 1 \times 6 = 12$

Update:

- $s \rightarrow a$: cap $1-1=0$
- $a \rightarrow b$: cap $5-1=4$, $b \rightarrow a$: cap $0+1=1$, cost $-0=0$
- $b \rightarrow t$: cap $2-1=1$

Phase 3: Shortest path

Residual graph:

- $s \rightarrow b$: cap 2, cost 3
- $a \rightarrow s$: cap 2, cost -2
- $t \rightarrow a$: cap 2, cost -1
- $a \rightarrow b$: cap 4, cost 0
- $b \rightarrow a$: cap 1, cost 0
- $b \rightarrow t$: cap 1, cost 4
- $t \rightarrow b$: cap 0, cost -4

Shortest path from s to t :

- $s \rightarrow b \rightarrow t$: cost = $3 + 4 = 7$
- $s \rightarrow b \rightarrow a \rightarrow s \rightarrow b \rightarrow t$: cycle
- $s \rightarrow \dots$ wait, can we go $s \rightarrow b \rightarrow a \rightarrow t$? $a \rightarrow t$ has cap 0, but we can go $t \rightarrow a$ (cap 2). So $s \rightarrow b \rightarrow a \rightarrow t$? No, $a \rightarrow t$ is 0.
- Can we go $s \rightarrow b \rightarrow a \rightarrow s \rightarrow b \rightarrow t$? $s \rightarrow b(3)$, $b \rightarrow a(0)$, $a \rightarrow s(-2)$, $s \rightarrow b(3)$, $b \rightarrow t(4) = 8$. Worse.

Shortest: $s \rightarrow b \rightarrow t$, cost = 7

Bottleneck: $\min(2, 1) = 1$ Flow = $3 + 1 = 4$, Cost = $12 + 1 \times 7 = 19$

Update:

- $s \rightarrow b$: cap $2-1=1$

- $b \rightarrow t$: cap $1-1=0$
- $t \rightarrow b$: cap $0+1=1$, cost -4

Phase 4: No path from s to t ($s \rightarrow b$ cap $1 > 0$, but $b \rightarrow t$ cap 0 , and no other path)

Flow = 4, Cost = 19

But wait, is this the max flow? Let me check: s has outgoing edges: $s \rightarrow a$ (cap 0), $s \rightarrow b$ (cap 1). So max flow = 4? Actually, $s \rightarrow b$ still has cap 1, but we can't reach t from b . So yes, max flow = 4.

Hmm, but I should check if there's a way: $s \rightarrow b \rightarrow a \rightarrow s \rightarrow a$??? No, $s \rightarrow a$ is 0. And $a \rightarrow t$ is 0. So no path.

Max flow = 4, Min cost = 19.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev;
    long long cap, cost;
};

class MinCostMaxFlow {
    int n;
    vector<vector<Edge>> adj;
    vector<long long> pot, dist;
    vector<int> parent, parentEdge;

    bool dijkstra(int s, int t) {
        dist.assign(n, LLONG_MAX);
        parent.assign(n, -1);
        parentEdge.assign(n, -1);

        priority_queue<pair<long long, int>, vector<pair<long long, int>>,
greater<>> pq;
        dist[s] = 0;
        pq.push({0, s});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;

            for (int i = 0; i < (int)adj[u].size(); i++) {
                auto& e = adj[u][i];
                if (e.cap == 0) continue;

                long long nd = d + e.cost + pot[u] - pot[e.to];
                if (nd < dist[e.to]) {
                    dist[e.to] = nd;
                    parent[e.to] = u;
                }
            }
        }
    }
};
```



```

        parentEdge[e.to] = i;
        pq.push({nd, e.to});
    }
}
}
return dist[t] != LLONG_MAX;
}

public:
    MinCostMaxFlow(int n) : n(n) {
        adj.resize(n);
        pot.assign(n, 0);
    }

    void addEdge(int u, int v, long long cap, long long cost) {
        Edge forward = {v, (int)adj[v].size(), cap, cost};
        Edge backward = {u, (int)adj[u].size(), 0, -cost};
        adj[u].push_back(forward);
        adj[v].push_back(backward);
    }

    // Returns {maxFlow, minCost}
    pair<long long, long long> minCostMaxFlow(int s, int t) {
        long long flow = 0, cost = 0;

        while (dijkstra(s, t)) {
            // Update potentials
            for (int i = 0; i < n; i++)
                if (dist[i] < LLONG_MAX)
                    pot[i] += dist[i];

            // Find bottleneck
            long long bottleneck = LLONG_MAX;
            int v = t;
            while (v != s) {
                int u = parent[v];
                int ei = parentEdge[v];
                bottleneck = min(bottleneck, adj[u][ei].cap);
                v = u;
            }

            // Push flow
            flow += bottleneck;
            cost += bottleneck * pot[t]; // pot[t] = shortest path distance from s
            to t

            v = t;
            while (v != s) {
                int u = parent[v];
                int ei = parentEdge[v];
                adj[u][ei].cap -= bottleneck;
                adj[v][adj[u][ei].rev].cap += bottleneck;
                v = u;
            }
        }
    }

```

```

    }

    return {flow, cost};
}
};

// Example usage
int main() {
    int n = 4;
    MinCostMaxFlow mcmf(n);
    mcmf.addEdge(0, 1, 3, 2); // s → a
    mcmf.addEdge(0, 2, 2, 3); // s → b
    mcmf.addEdge(1, 3, 2, 1); // a → t
    mcmf.addEdge(2, 3, 2, 4); // b → t
    mcmf.addEdge(1, 2, 5, 0); // a → b

    auto [flow, cost] = mcmf.minCostMaxFlow(0, 3);
    cout << "Max flow: " << flow << "\n";
    cout << "Min cost: " << cost << "\n";

    return 0;
}

```

9. Python Implementation

```

import heapq

class MinCostMaxFlow:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.pot = [0] * n

    def add_edge(self, u, v, cap, cost):
        self.adj[u].append([v, cap, cost, len(self.adj[v])])
        self.adj[v].append([u, 0, -cost, len(self.adj[u]) - 1])

    def _dijkstra(self, s, t):
        n = self.n
        dist = [float('inf')] * n
        parent = [-1] * n
        parent_edge = [-1] * n

        dist[s] = 0
        pq = [(0, s)]

        while pq:
            d, u = heapq.heappop(pq)
            if d != dist[u]:
                continue
            for i, (v, cap, cost, _) in enumerate(self.adj[u]):

```

```

        if cap == 0:
            continue
        nd = d + cost + self.pot[u] - self.pot[v]
        if nd < dist[v]:
            dist[v] = nd
            parent[v] = u
            parent_edge[v] = i
            heapq.heappush(pq, (nd, v))

    if dist[t] == float('inf'):
        return None, None, None

    # Update potentials
    for i in range(n):
        if dist[i] < float('inf'):
            self.pot[i] += dist[i]

    # Find bottleneck
    bottleneck = float('inf')
    v = t
    while v != s:
        u = parent[v]
        ei = parent_edge[v]
        bottleneck = min(bottleneck, self.adj[u][ei][1])
        v = u

    return parent, parent_edge, bottleneck

def min_cost_max_flow(self, s, t):
    flow = 0
    cost = 0

    while True:
        parent, parent_edge, bottleneck = self._dijkstra(s, t)
        if parent is None:
            break

        flow += bottleneck
        cost += bottleneck * self.pot[t]

        v = t
        while v != s:
            u = parent[v]
            ei = parent_edge[v]
            self.adj[u][ei][1] -= bottleneck
            rev = self.adj[u][ei][3]
            self.adj[v][rev][1] += bottleneck
            v = u

    return flow, cost

```

10. Code Explanation

Potentials (Johnson's technique):

- Initial potentials: 0 (assuming no negative edges initially).
- After each Dijkstra run, update $\text{pot}[v] += \text{dist}[v]$.
- Reduced cost: $\text{cost} + \text{pot}[u] - \text{pot}[v]$ is always non-negative.
- This allows using Dijkstra ($O(E \log V)$) instead of Bellman-Ford ($O(VE)$).

Dijkstra:

- Uses reduced costs (adjusted by potentials).
- Returns parent pointers for reconstructing the path.
- Returns bottleneck capacity.

Main Loop:

- Run Dijkstra to find shortest augmenting path.
- If no path, break.
- Push bottleneck flow along the path.
- Update residual capacities.
- Update total flow and cost.

Edge Structure:

- Same as Dinic: to , rev , cap , cost .
- Reverse edge has negative cost.

11. Complexity Analysis

Operation	Time	Space
Each Dijkstra	$O(E \log V)$	$O(V)$
Number of iterations	$O(F)$ ($F = \text{max flow}$)	-
Total	$O(F \times E \log V)$	$O(V + E)$
With scaling	$O(U \times E \log V)$ ($U = \text{max capacity}$)	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Minimum cost bipartite matching	Add source $\rightarrow U$, $U \rightarrow V$, $V \rightarrow \text{sink}$ with costs	"minimum cost matching"	GFG
Transportation problem	Supply nodes, demand nodes, shipping costs	"minimize transportation cost"	Various
Minimum cost flow with demand	Node demands instead of source/sink	"circulation with demands"	Codeforces
Job scheduling	Jobs with time windows and costs	"schedule jobs minimizing cost"	UVA

13. Common Mistakes

- **Not updating potentials:** Potentials must be updated after each Dijkstra run.
- **Using Bellman-Ford instead of potentials:** Dijkstra with potentials is much faster.
- **Integer overflow:** Use `long long` for costs and flows.
- **Negative cycles:** The algorithm assumes no negative cost cycles. If present, use Bellman-Ford first.
- **Costs on reverse edges:** Reverse edges must have negative cost of forward edges.

14. Edge Cases

- **No path from s to t:** Flow = 0, cost = 0.
- **All costs zero:** Reduces to max flow (use Dinic instead).
- **Negative costs:** Potentials handle this, but initial potentials must be set correctly.
- **Large capacities:** Use `long long`.
- **Disconnected graph:** Returns immediately.

15. Variations

Variation	What Changes	When Used	Importance
Successive Shortest Path	Dijkstra with potentials (standard)	General MCMF	High
Capacity Scaling	Process large capacities first	Large capacity ranges	Medium
Cost Scaling	Scale costs to find approximate solution	Very large graphs	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Dinic	Max flow only (no costs)	Use Dinic if no costs involved
Hungarian Algorithm	Assignment problem specifically	Use Hungarian for balanced assignment
Bellman-Ford	Can find shortest paths with negative edges	Use Bellman-Ford if negative edges exist without potentials

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Minimum Cost Flow	GFG	Basic MCMF	Easy
Minimum Cost to Make at Least One Valid Path	LeetCode	Simple MCMF	Easy

Medium

Problem	Platform	Idea	Difficulty
Minimum Cost Flow	CSES	MCMF template	Medium
Transportation Problem	Codeforces	Supply-demand with costs	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Cost Maximum Flow	Codeforces	MCMF on large graph	Hard
Job Scheduling with Costs	AtCoder	Schedule optimization	Hard

18. Interview Explanation

"Min-cost max-flow finds the cheapest way to send maximum flow from source to sink. We use the successive shortest augmenting path algorithm: each iteration, we find the shortest path in the residual graph using edge costs, push as much flow as possible, and update. To handle negative costs from reverse edges, we use Johnson's potentials to keep reduced costs non-negative, allowing Dijkstra instead of Bellman-Ford. The algorithm runs in $O(F \cdot E \log V)$ where F is the max flow value."

19. Revision Notes

- **Goal:** Max flow with minimum total cost.
- **Algorithm:** Successive shortest augmenting path (Dijkstra with potentials).
- **Potentials:** `pot[v] += dist[v]` after each Dijkstra.
- **Reverse edges:** Negative cost of forward edge.
- **Complexity:** $O(F \times E \log V)$.
- **Trap:** Must update potentials correctly.

20. Final Cheat Sheet

Aspect	Details
When to use	Flow with costs, transportation, min-cost matching
Main operations	Dijkstra with potentials, augment path, update residual
Time complexity	$O(F \times E \log V)$
Space complexity	$O(V + E)$
Key code idea	<code>pot[v] += dist[v]</code> ; reduced cost = <code>cost + pot[u] - pot[v]</code> ; reverse edge cost = <code>-cost</code>
Edge cases	No path (flow=0), zero costs (use Dinic), negative costs

10. HOPCROFT-KARP ALGORITHM

1. Overview

Hopcroft-Karp is an algorithm for finding the maximum matching in a bipartite graph. It is significantly faster than the simple DFS-based augmenting path algorithm, achieving $O(E\sqrt{V})$ time instead of $O(V \times E)$.

2. Intuition

Simple Explanation

Instead of finding one augmenting path at a time (like the simple DFS approach), Hopcroft-Karp finds **multiple vertex-disjoint shortest augmenting paths** in one BFS/DFS phase. This reduces the number of phases to $O(\sqrt{V})$.

Analogy

Imagine you're matching mentors to mentees. Instead of finding one match at a time (simple DFS), Hopcroft-Karp finds all the shortest possible matches simultaneously using a BFS to find the shortest path length, then a DFS to find all vertex-disjoint paths of that length.

Step-by-Step Reasoning

1. **BFS Phase:** Build a layered graph from all unmatched vertices in U to all unmatched vertices in V . Assign levels (distances). Only consider augmenting paths.
2. **DFS Phase:** Find all vertex-disjoint augmenting paths of the current shortest length. Use DFS with the level graph.
3. Each phase finds a maximal set of vertex-disjoint shortest augmenting paths.
4. The shortest augmenting path length increases by at least 2 after each phase.
5. After $O(\sqrt{V})$ phases, the matching is maximum.

Why It Works

The key insight: the shortest augmenting path length increases by at least 2 after each phase. Since the maximum possible path length is $O(V)$, there are at most $O(\sqrt{V})$ phases (because each phase increases the shortest path length, and the total number of phases is bounded by $O(\sqrt{V})$ due to a more complex analysis).

3. When to Use It

- **Maximum bipartite matching:** The fastest general-purpose algorithm.
- **Large graphs:** Where $O(V \times E)$ is too slow.
- **Competitive programming:** When constraints are high (e.g., V, E up to 10^5).
- **Dense graphs:** Where $V \times E$ would be prohibitive.

4. When Not to Use It

- **Small graphs:** Simple DFS augmenting path is easier to implement and fast enough.
- **Weighted matching:** Use Hungarian algorithm.
- **Non-bipartite graphs:** Use Edmonds' Blossom algorithm.

- **Maximum flow with additional constraints:** Use Dinic.

5. Core Concepts

5.1 Distance Layer

BFS from all unmatched U vertices to all unmatched V vertices. Each node gets a distance (level).

5.2 Vertex-Disjoint Paths

Paths that share no vertices (except possibly the start and end).

5.3 Phase

One BFS + one DFS (finding maximal set of vertex-disjoint shortest augmenting paths).

5.4 NIL (Dummy Node)

A sentinel node representing "unmatched." Used to simplify BFS termination.

6. Step-by-Step Algorithm

1. Initialize `pairU[u] = NIL`, `pairV[v] = NIL` for all vertices.
2. While BFS finds an augmenting path: a. **BFS**: From all unmatched U vertices, build level graph until we reach an unmatched V vertex. b. **DFS**: For each unmatched U vertex, try to find an augmenting path using the level graph. If found, update matching.
3. Return matching size.

7. Dry Run

$U = \{0, 1, 2\}$, $V = \{3, 4, 5\}$ Edges: 0-3, 0-4, 1-3, 1-5, 2-4

Phase 1

BFS:

- Start: $\{0, 1, 2\}$ (all unmatched)
- Layer 0: $\{0, 1, 2\}$
- Layer 1: $\{3, 4, 5\}$ (neighbors of 0, 1, 2)
 - 3: from 0, 1
 - 4: from 0, 2
 - 5: from 1

Since we reached V vertices, BFS stops. $\text{dist}=1$.

DFS:

- DFS(0): try 3 (unmatched) → match 0-3
- DFS(1): try 3 (matched to 0), try 0 (unmatched? no, 0 is matched but we're looking for augmenting path)
 - $1 \rightarrow 3 \rightarrow 0$, then $0 \rightarrow 4$ (unmatched) → match 1-3, 0-4
 - Wait, this is getting complex. Let me trace more carefully.

Actually, the DFS in Hopcroft-Karp works like this:

- For each unmatched U vertex, run DFS that only goes to next level.
- If we reach an unmatched V vertex, we've found an augmenting path.

DFS for u=0:

- 0→3 (level 1, pairV[3]=NIL): found! match 0-3
- pairU[0]=3, pairV[3]=0

DFS for u=1:

- 1→3 (level 1, pairV[3]=0, not NIL): try DFS(0)
 - 0→4 (level 1, pairV[4]=NIL): found! match 0-4
 - Back: match 1-3
- pairU[1]=3, pairV[3]=1, pairU[0]=4, pairV[4]=0

DFS for u=2:

- 2→4 (level 1, pairV[4]=0, not NIL): try DFS(0)
 - 0→3 (level 1, pairV[3]=1, not NIL): try DFS(1)
 - 1→5 (level 1, pairV[5]=NIL): found! match 1-5
 - Back: match 0-3
 - Back: match 2-4
- pairU[2]=4, pairV[4]=2, pairU[0]=3, pairV[3]=0, pairU[1]=5, pairV[5]=1

Phase 1 done. Matching: (0-3, 1-5, 2-4), size = 3.

Phase 2

BFS: All U vertices are matched. No unmatched U vertex. BFS returns NIL distance. Done.

Max matching size = 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class HopcroftKarp {
    int n, m; // sizes of U and V
    vector<vector<int>>> adj; // adjacency from U to V
    vector<int> pairU, pairV, dist;
    const int NIL = -1;

    bool bfs() {
        queue<int> q;
        for (int u = 0; u < n; u++) {
            if (pairU[u] == NIL) {
                dist[u] = 0;
                q.push(u);
            } else {

```

```

        dist[u] = INT_MAX;
    }
}
dist[NIL] = INT_MAX; // sentinel

while (!q.empty()) {
    int u = q.front(); q.pop();
    if (dist[u] < dist[NIL]) {
        for (int v : adj[u]) {
            // Go to the vertex matched to v (or NIL if unmatched)
            int nextU = pairV[v];
            if (dist[nextU] == INT_MAX) {
                dist[nextU] = dist[u] + 1;
                q.push(nextU);
            }
        }
    }
}

return dist[NIL] != INT_MAX;
}

bool dfs(int u) {
    if (u == NIL) return true;
    for (int v : adj[u]) {
        int nextU = pairV[v];
        if (dist[nextU] == dist[u] + 1 && dfs(nextU)) {
            pairU[u] = v;
            pairV[v] = u;
            return true;
        }
    }
    dist[u] = INT_MAX;
    return false;
}

public:
    HopcroftKarp(int n, int m, vector<vector<int>>& adj)
        : n(n), m(m), adj(adj) {
        pairU.assign(n, NIL);
        pairV.assign(m, NIL);
        dist.assign(n + 1, 0); // extra slot for NIL (index n)
    }

    int maxMatching() {
        int matching = 0;
        while (bfs()) {
            for (int u = 0; u < n; u++) {
                if (pairU[u] == NIL && dfs(u)) {
                    matching++;
                }
            }
        }
        return matching;
    }
}

```

```

    vector<int> getMatching() { return pairU; }
};

// Example usage
int main() {
    int n = 3, m = 3;
    vector<vector<int>> adj = {
        {0, 1},    // U0
        {0, 2},    // U1
        {1}        // U2
    };

    HopcroftKarp hk(n, m, adj);
    int maxMatch = hk.maxMatching();
    cout << "Maximum matching size: " << maxMatch << "\n";

    auto matching = hk.getMatching();
    for (int u = 0; u < n; u++) {
        cout << "U" << u << " -> V" << matching[u] << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

from collections import deque

class HopcroftKarp:
    def __init__(self, n, m, adj):
        self.n = n # size of U
        self.m = m # size of V
        self.adj = adj # adjacency from U to V
        self.pairU = [-1] * n
        self.pairV = [-1] * m
        self.dist = [0] * (n + 1)
        self.NIL = -1

    def _bfs(self):
        q = deque()
        for u in range(self.n):
            if self.pairU[u] == self.NIL:
                self.dist[u] = 0
                q.append(u)
            else:
                self.dist[u] = float('inf')
        self.dist[self.NIL] = float('inf')

        while q:
            u = q.popleft()

```

```

        if self.dist[u] < self.dist[self.NIL]:
            for v in self.adj[u]:
                nextU = self.pairV[v]
                if self.dist[nextU] == float('inf'):
                    self.dist[nextU] = self.dist[u] + 1
                    q.append(nextU)
            return self.dist[self.NIL] != float('inf')

def _dfs(self, u):
    if u == self.NIL:
        return True
    for v in self.adj[u]:
        nextU = self.pairV[v]
        if self.dist[nextU] == self.dist[u] + 1 and self._dfs(nextU):
            self.pairU[u] = v
            self.pairV[v] = u
            return True
    self.dist[u] = float('inf')
    return False

def max_matching(self):
    matching = 0
    while self._bfs():
        for u in range(self.n):
            if self.pairU[u] == self.NIL and self._dfs(u):
                matching += 1
    return matching

def get_matching(self):
    return self.pairU

```

10. Code Explanation

BFS:

- Starts from all unmatched U vertices (distance 0).
- For each U vertex, explores its V neighbors.
- From a V vertex, goes to the U vertex matched to it (or NIL if unmatched).
- `dist[NIL]` is the distance to the nearest unmatched V vertex.
- Returns true if an augmenting path exists.

DFS:

- Follows the level graph (only edges where `dist[nextU] == dist[u] + 1`).
- If we reach NIL (unmatched V vertex), we've found an augmenting path.
- Updates matching on the way back.
- If no path found, sets `dist[u] = INF` to avoid re-exploring.

Main Loop:

- While BFS finds augmenting paths:
 - For each unmatched U vertex, try DFS.

- Each successful DFS increases matching size by 1.

11. Complexity Analysis

Operation	Time	Space
Each BFS/DFS phase	$O(E)$	$O(V)$
Number of phases	$O(\sqrt{V})$	-
Total	$O(E\sqrt{V})$	$O(V + E)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Maximum bipartite matching	Standard matching	"maximum matching"	GFG, CSES
Minimum vertex cover	König's theorem	"minimum vertex cover"	Various
Maximum independent set	Total - matching	"maximum independent set"	Various
Minimum path cover in DAG	Transform to bipartite graph	"minimum path cover"	Codeforces

13. Common Mistakes

- **Not handling NIL correctly:** NIL is a sentinel; its distance must be initialized correctly.
- **Incorrect BFS termination:** BFS should stop when we reach an unmatched V vertex (NIL).
- **DFS not checking level:** DFS must only follow edges where `dist[nextU] == dist[u] + 1`.
- **Not resetting dist for unmatched U vertices:** Each BFS must start from all unmatched U vertices.
- **Index confusion:** V vertices might be 0-indexed or n-indexed.

14. Edge Cases

- **Empty graph:** Matching size = 0.
- **One side empty:** Matching size = 0.
- **Complete bipartite graph:** Matching size = $\min(n, m)$.
- **Already perfect matching:** Algorithm terminates after one phase.

15. Variations

Variation	What Changes	When Used	Importance
Dinic for bipartite matching	Use Dinic instead of Hopcroft-Karp	Also $O(E\sqrt{V})$, simpler implementation	High
Simple DFS augmenting path	$O(V \times E)$	Small graphs	Medium

Variation	What Changes	When Used	Importance
Hungarian Algorithm	Weighted matching	When edges have weights	High

16. Related Algorithms

Algorithm	Connection	How to Choose
Simple DFS Matching	Simpler, $O(V \times E)$	Small graphs ($V < 1000$)
Dinic	Also $O(E\sqrt{V})$ for bipartite matching	Use Dinic if you already have max flow code
Hungarian	Weighted version	Use when edges have costs
Blossom (Edmonds)	Non-bipartite matching	Use for general graphs

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Maximum Bipartite Matching	GFG	Basic Hopcroft-Karp	Easy
Matching	CSES	Maximum matching	Easy

Medium

Problem	Platform	Idea	Difficulty
Minimum Path Cover	Codeforces	Transform to bipartite matching	Medium
Domino Placement	Codeforces	Grid bipartite matching	Medium

Hard

Problem	Platform	Idea	Difficulty
Maximum Matching on General Graph	Codeforces	Blossom algorithm	Hard
Bipartite Matching with Large Constraints	Codeforces	Optimized Hopcroft-Karp	Hard

18. Interview Explanation

"Hopcroft-Karp finds maximum bipartite matching in $O(E\sqrt{V})$ time. It works in phases: each phase uses BFS to find the shortest augmenting path length, then uses DFS to find all vertex-disjoint augmenting paths of that length. The key insight is that the shortest path length increases by at least 2 after each phase, and there are at most $O(\sqrt{V})$ phases. This is much faster than the simple $O(V \times E)$ DFS approach."

19. Revision Notes

- **Goal:** Maximum bipartite matching in $O(E\sqrt{V})$.
- **BFS:** Build level graph from unmatched U vertices.
- **DFS:** Find vertex-disjoint shortest augmenting paths.
- **Phases:** $O(\sqrt{V})$ phases, each $O(E)$.
- **Trap:** Use NIL sentinel correctly; DFS must respect level graph.

20. Final Cheat Sheet

Aspect	Details
When to use	Maximum bipartite matching, large graphs, CP
Main operations	BFS (level graph) + DFS (vertex-disjoint paths)
Time complexity	$O(E\sqrt{V})$
Space complexity	$O(V + E)$
Key code idea	BFS from unmatched U; DFS with level constraints; NIL sentinel
Edge cases	Empty graph, complete bipartite, already perfect matching

11. 2-SAT (2-SATISFIABILITY)

1. Overview

2-SAT is a problem of determining whether a Boolean formula in Conjunctive Normal Form (CNF) with exactly 2 literals per clause is satisfiable. It is a special case of the SAT problem that can be solved in polynomial time (in fact, linear time) using strongly connected components.

2. Intuition

Simple Explanation

You have a set of Boolean variables, and you need to assign each variable true/false such that all given constraints are satisfied. Each constraint is of the form $(A \text{ OR } B)$, where A and B are variables or their negations. For example, $(x \text{ OR } y)$ and $(\text{not } x \text{ OR } z)$.

Analogy

Think of a scheduling problem: each of two events can be held either in the morning or afternoon. Constraints like "at least one of event A or event B must be in the morning" or "if event C is in the afternoon, event D must also be in the afternoon" are 2-SAT constraints.

Step-by-Step Reasoning

1. Each clause $(A \text{ OR } B)$ is equivalent to $(\text{not } A \rightarrow B)$ and $(\text{not } B \rightarrow A)$.
2. Build an implication graph: each variable has two nodes (true and false), and each clause adds two directed edges.

3. If there is a path from x to $\text{not } x$ AND from $\text{not } x$ to x , then x must be both true and false $\rightarrow$ unsatisfiable.
4. In other words, if x and $\text{not } x$ are in the same SCC, the formula is unsatisfiable.
5. Otherwise, assign truth values based on topological order of SCCs.

Why It Works

The implication graph captures all logical implications. If $x \rightarrow y$ is in the graph, then whenever x is true, y must also be true. If there's a cycle containing both x and $\neg x$, then x implies $\neg x$ and $\neg x$ implies x , which is contradictory. Otherwise, we can assign values by processing SCCs in reverse topological order: assign false to the first SCC we encounter, which forces its negation to be true.

3. When to Use It

- **Boolean satisfiability with 2 literals per clause:** The classic application.
- **Constraint satisfaction problems:** Where each constraint involves at most 2 variables.
- **Scheduling problems:** Two time slots, each event must be in one slot, with pairwise constraints.
- **Graph coloring with 2 colors:** Each constraint says two vertices must have different colors.
- **Circuit design:** With binary decisions.
- **Maze/ puzzle problems:** Where each cell has two possible states.

Common trigger phrases:

- "SAT"
- "2-SAT"
- "satisfiability"
- "each variable is either true or false"
- "constraints of the form (A OR B)"
- "2-CNF"

4. When Not to Use It

- **General SAT (3-SAT or more):** NP-complete. Use SAT solvers.
- **No clear implication structure:** Other constraint satisfaction techniques might be better.
- **More than 2 literals per clause:** Use general SAT or other techniques.
- **Non-Boolean variables:** Use CSP solvers.

5. Core Concepts

5.1 Literal

A variable or its negation: x or $\neg x$.

5.2 Clause

A disjunction (OR) of literals. In 2-SAT, each clause has exactly 2 literals.

5.3 CNF (Conjunctive Normal Form)

A conjunction (AND) of clauses. The formula is satisfiable if all clauses are satisfied.

5.4 Implication Graph

A directed graph with $2n$ nodes (one for each literal and its negation). Edge $a \rightarrow b$ means "if a is true, then b must be true."

5.5 Implication

$(A \text{ OR } B)$ is equivalent to $(\neg A \rightarrow B)$ and $(\neg B \rightarrow A)$.

6. Step-by-Step Algorithm

1. **Build implication graph:** For each clause $(A \text{ OR } B)$, add edges $\neg A \rightarrow B$ and $\neg B \rightarrow A$.
2. **Find SCCs:** Use Kosaraju or Tarjan to find SCCs in the implication graph.
3. **Check satisfiability:** If any variable x and its negation $\neg x$ are in the same SCC, formula is unsatisfiable.
4. **Assign truth values:** Process SCCs in reverse topological order. For each SCC, if its variables are not yet assigned, assign false to all literals in the SCC (which means the corresponding variables are true/false as needed).

A simpler assignment: x is true if $\text{comp}[x] > \text{comp}[\text{not } x]$ (where comp is the SCC ID in topological order).

7. Dry Run

Formula: $(x \text{ OR } y) \wedge (\neg x \text{ OR } y) \wedge (x \text{ OR } \neg y)$

Variables: x, y . Clauses:

1. $(x \text{ OR } y)$
2. $(\neg x \text{ OR } y)$
3. $(x \text{ OR } \neg y)$

Implication Graph

Variables: $x, \neg x, y, \neg y$

Clause 1 ($x \text{ OR } y$): $\neg x \rightarrow y, \neg y \rightarrow x$ Clause 2 ($\neg x \text{ OR } y$): $x \rightarrow y, \neg y \rightarrow \neg x$ Clause 3 ($x \text{ OR } \neg y$): $\neg x \rightarrow \neg y, y \rightarrow x$

Edges:

- $\neg x \rightarrow y$
- $\neg y \rightarrow x$
- $x \rightarrow y$
- $\neg y \rightarrow \neg x$
- $\neg x \rightarrow \neg y$
- $y \rightarrow x$

SCC Detection

Let's find SCCs:

From x : $x \rightarrow y, y \rightarrow x \rightarrow \text{cycle}$. $x \rightarrow y \rightarrow x$ forms SCC $\{x, y\}$ From $\neg x$: $\neg x \rightarrow y, \neg x \rightarrow \neg y, \neg y \rightarrow \neg x, \neg y \rightarrow x$. So $\neg x \rightarrow \neg y \rightarrow \neg x$ forms SCC $\{\neg x, \neg y\}$

And also: $\neg x \rightarrow y$ (but y is in $\{x, y\}$), $\neg y \rightarrow x$ (x is in $\{x, y\}$)

So we have two SCCs: $\{x, y\}$ and $\{\neg x, \neg y\}$

Check Satisfiability

x and $\neg x$ are in different SCCs ✓ y and $\neg y$ are in different SCCs ✓

Satisfiable!

Assignment

SCCs in topological order (reverse of Kosaraju order):

- If $\text{comp}[\neg x]$ comes before $\text{comp}[x]$, assign $x = \text{true}$.
- Let's say $\text{comp}[\neg x] = 0$, $\text{comp}[x] = 1$.
- $\text{comp}[\neg x] < \text{comp}[x] \rightarrow x = \text{true}$.
- $\text{comp}[\neg y] = 0$, $\text{comp}[y] = 1 \rightarrow y = \text{true}$.

Check: $(x \text{ OR } y) = (T \text{ OR } T) = T$ ✓, $(\neg x \text{ OR } y) = (F \text{ OR } T) = T$ ✓, $(x \text{ OR } \neg y) = (T \text{ OR } F) = T$ ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class TwoSAT {
    int n; // number of variables
    vector<vector<int>> adj, radj;
    vector<int> comp;
    vector<bool> visited;
    vector<int> order;

    void dfs1(int u) {
        visited[u] = true;
        for (int v : adj[u])
            if (!visited[v])
                dfs1(v);
        order.push_back(u);
    }

    void dfs2(int u, int cid) {
        comp[u] = cid;
        for (int v : radj[u])
            if (comp[v] == -1)
                dfs2(v, cid);
    }

    // literal to node index:  $x \rightarrow 2*x$ ,  $\neg x \rightarrow 2*x+1$ 
    int node(int var, bool isNeg) {
        return 2 * var + (isNeg ? 1 : 0);
    }
}
```

```

int neg(int u) {
    return u ^ 1; // flip last bit: 2x ⇔ 2x+1
}

public:
    TwoSAT(int n) : n(n) {
        adj.resize(2 * n);
        radj.resize(2 * n);
    }

    // Add clause (a OR b)
    // a, b: variable index (0-indexed)
    // aNeg, bNeg: whether the literal is negated
    void addClause(int a, bool aNeg, int b, bool bNeg) {
        int u = node(a, aNeg);
        int v = node(b, bNeg);
        // ¬a → b
        adj[neg(u)].push_back(v);
        radj[v].push_back(neg(u));
        // ¬b → a
        adj[neg(v)].push_back(u);
        radj[u].push_back(neg(v));
    }

    // Returns true if satisfiable, and assigns values
    bool solve(vector<bool>& result) {
        int N = 2 * n;
        visited.assign(N, false);
        order.clear();

        // Kosaraju first pass
        for (int i = 0; i < N; i++)
            if (!visited[i])
                dfs1(i);

        comp.assign(N, -1);
        int cid = 0;
        for (int i = N - 1; i >= 0; i--) {
            int u = order[i];
            if (comp[u] == -1)
                dfs2(u, cid++);
        }

        // Check satisfiability
        for (int i = 0; i < n; i++) {
            if (comp[node(i, false)] == comp[node(i, true)])
                return false;
        }

        // Assign truth values
        result.assign(n, false);
        for (int i = 0; i < n; i++) {
            // If comp[¬x] > comp[x], x is true (later in topological order =

```

```

smaller comp ID)
    // Actually, in Kosaraju, comp ID is assigned in reverse topological
order.

    // Higher comp ID = earlier in topological order.
    // We assign x = true if comp[x] < comp[¬x]
    result[i] = comp[node(i, false)] < comp[node(i, true)];
}

return true;
}
};

// Example usage
int main() {
    // Formula: (x OR y) ∧ (¬x OR y) ∧ (x OR ¬y)
    TwoSAT sat(2); // 2 variables: x=0, y=1

    sat.addClause(0, false, 1, false); // (x OR y)
    sat.addClause(0, true, 1, false);  // (¬x OR y)
    sat.addClause(0, false, 1, true);  // (x OR ¬y)

    vector<bool> result;
    if (sat.solve(result)) {
        cout << "Satisfiable!\n";
        cout << "x = " << (result[0] ? "true" : "false") << "\n";
        cout << "y = " << (result[1] ? "true" : "false") << "\n";
    } else {
        cout << "Unsatisfiable!\n";
    }

    return 0;
}

```

9. Python Implementation

```

class TwoSAT:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(2 * n)]
        self.radj = [[] for _ in range(2 * n)]

    def _node(self, var, is_neg):
        return 2 * var + (1 if is_neg else 0)

    def _neg(self, u):
        return u ^ 1

    def add_clause(self, a, a_neg, b, b_neg):
        u = self._node(a, a_neg)
        v = self._node(b, b_neg)
        # ¬a → b

```

```

self.adj[self._neg(u)].append(v)
self.radj[v].append(self._neg(u))
# -b → a
self.adj[self._neg(v)].append(u)
self.radj[u].append(self._neg(v))

def solve(self):
    N = 2 * self.n

    # Kosaraju first pass
    visited = [False] * N
    order = []

    def dfs1(u):
        visited[u] = True
        for v in self.adj[u]:
            if not visited[v]:
                dfs1(v)
        order.append(u)

    for i in range(N):
        if not visited[i]:
            dfs1(i)

    # Kosaraju second pass
    comp = [-1] * N
    cid = 0

    def dfs2(u, cid):
        comp[u] = cid
        for v in self.radj[u]:
            if comp[v] == -1:
                dfs2(v, cid)

    for u in reversed(order):
        if comp[u] == -1:
            dfs2(u, cid)
            cid += 1

    # Check satisfiability
    for i in range(self.n):
        if comp[self._node(i, False)] == comp[self._node(i, True)]:
            return False, []

    # Assign values
    result = [False] * self.n
    for i in range(self.n):
        result[i] = comp[self._node(i, False)] < comp[self._node(i, True)]

    return True, result

```

10. Code Explanation

Node Mapping:

- Variable x (0-indexed) $\rightarrow$ node $2x$ (positive literal), $2x+1$ (negative literal).
- $\text{neg}(u)$: flips last bit ($2x \leftrightarrow 2x+1$).

Adding Clause:

- Clause $(a \text{ OR } b) \rightarrow$ edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$.
- Both forward and reverse edges for Kosaraju.

SCC Detection:

- Standard Kosaraju on the implication graph ($2n$ nodes).

Satisfiability Check:

- For each variable x , check if $\text{comp}[x] == \text{comp}[\neg x]$.
- If any such pair exists, formula is unsatisfiable.

Assignment:

- In Kosaraju, comp IDs are assigned in reverse topological order of the condensation DAG.
- Higher comp ID = earlier in topological order = should be assigned earlier.
- We assign $x = \text{true}$ if $\text{comp}[x] < \text{comp}[\neg x]$ (x is later in topological order, so $\neg x$ is assigned false first).

11. Complexity Analysis

Operation	Time	Space
Building graph	$O(C)$ (C = number of clauses)	$O(C)$
Kosaraju SCC	$O(V + E) = O(n + C)$	$O(n + C)$
Assignment	$O(n)$	$O(n)$
Total	$O(n + C)$	$O(n + C)$

12. Common Patterns

Pattern	Description	How to Identify	Example
Standard 2-SAT	Clauses with 2 literals	"(A OR B)" constraints	CSES
Exactly one of two	$(A \text{ OR } B) \wedge (\neg A \text{ OR } \neg B)$	"exactly one"	Various
Implication constraints	$A \rightarrow B$ means $(\neg A \text{ OR } B)$	"if A then B"	Codeforces
Equivalence	$A \leftrightarrow B$ means $(\neg A \text{ OR } B) \wedge (A \text{ OR } \neg B)$	"A if and only if B"	Various

13. Common Mistakes

- Wrong node indexing:** Variable $x \rightarrow$ node $2x$ (positive) and $2x+1$ (negative). Off-by-one errors are common.

- **Forgetting both implications:** Clause (A OR B) needs both $\neg A \rightarrow B$ and $\neg B \rightarrow A$.
- **Not handling negation correctly:** $\neg(\neg x)$ is x .
- **Assignment logic:** The assignment rule $\text{comp}[x] < \text{comp}[\neg x] \rightarrow x = \text{true}$ depends on the specific SCC algorithm. For Kosaraju, higher comp ID = earlier in topological order.
- **Not checking both directions:** Must check both $\text{comp}[x] == \text{comp}[\neg x]$ for each variable.

14. Edge Cases

- **No clauses:** Always satisfiable.
- **Single clause:** Always satisfiable (assign appropriately).
- **Contradictory clauses:** $(x) \wedge (\neg x) \rightarrow$ unsatisfiable.
- **All variables constrained:** Each variable appears in many clauses.
- **Redundant clauses:** Same clause added multiple times.

15. Variations

Variation	What Changes	When Used	Importance
3-SAT	3 literals per clause (NP-complete)	Not polynomial	N/A
MAX-2-SAT	Find maximum number of satisfiable clauses	Optimization version	Medium
2-SAT with counting	Count number of satisfying assignments	Advanced	Low
Online 2-SAT	Add clauses incrementally	Dynamic constraints	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
SCC (Kosaraju/Tarjan)	Core component of 2-SAT	Use Kosaraju or Tarjan for SCC detection
General SAT	NP-complete	Use SAT solvers for 3-SAT+
Implication Graph	Directed graph of implications	Build from 2-CNF clauses

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
2-SAT	CSES	Basic 2-SAT	Easy
2-SAT (Basic)	GFG	Check satisfiability	Easy

Medium

Problem	Platform	Idea	Difficulty
2-SAT with 3 variables	Codeforces	Standard 2-SAT	Medium
Schedule	Codeforces	2-SAT for scheduling	Medium

Hard

Problem	Platform	Idea	Difficulty
2-SAT with Counting	Codeforces	Count satisfying assignments	Hard
Planar 2-SAT	Codeforces	2-SAT on planar graphs	Hard

18. Interview Explanation

"2-SAT is a polynomial-time solvable case of the SAT problem. We convert each clause (A OR B) into two implications: $\neg A \rightarrow B$ and $\neg B \rightarrow A$. We build an implication graph with $2n$ nodes (one for each literal and its negation). Then we find SCCs using Kosaraju's algorithm. If any variable and its negation are in the same SCC, the formula is unsatisfiable. Otherwise, we assign truth values based on the topological order of SCCs: if $\text{comp}[x] < \text{comp}[\neg x]$, x is true. The algorithm runs in $O(n + C)$ time."

19. Revision Notes

- **Clause $\rightarrow$ Implication:** $(A \text{ OR } B) \rightarrow (\neg A \rightarrow B) \wedge (\neg B \rightarrow A)$.
- **Implication graph:** $2n$ nodes, edges from implications.
- **Satisfiability check:** x and $\neg x$ in same SCC $\rightarrow$ unsatisfiable.
- **Assignment:** $\text{comp}[x] < \text{comp}[\neg x] \rightarrow x = \text{true}$ (for Kosaraju).
- **Complexity:** $O(n + C)$ linear.
- **Trap:** Both implications needed; correct node indexing ($2x, 2x+1$).

20. Final Cheat Sheet

Aspect	Details
When to use	Boolean constraints with 2 literals per clause, scheduling, implication problems
Main operations	Build implication graph, find SCCs, assign values
Time complexity	$O(n + C)$
Space complexity	$O(n + C)$
Key code idea	<code>addClause(a, aNeg, b, bNeg)</code> : add $\neg a \rightarrow b$ and $\neg b \rightarrow a$; check <code>comp[2x] == comp[2x+1]</code> for conflict
Edge cases	No clauses (satisfiable), contradictory clauses (unsatisfiable)

Final Notes

Quick Reference Table

Algorithm	Problem	Time Complexity	Key Technique
Kosaraju SCC	SCC in directed graph	$O(V + E)$	Two DFS passes, reversed graph
Tarjan SCC	SCC in directed graph	$O(V + E)$	Single DFS, low-link values
Bridges	Critical edges in undirected graph	$O(V + E)$	DFS with low-link, $low[v] > disc[u]$
Articulation Points	Critical vertices in undirected graph	$O(V + E)$	DFS with low-link, $low[v] \geq disc[u]$
Euler Path/Circuit	Path using every edge once	$O(V + E)$	Hierholzer's algorithm, degree check
Bipartite Matching	Max matching in bipartite graph	$O(V \times E) / O(E\sqrt{V})$	Augmenting paths
Max Flow (Edmonds-Karp)	Max flow in network	$O(V \times E^2)$	BFS augmenting paths
Dinic	Max flow in network	$O(V^2 \times E)$	Level graph + blocking flow
Min-Cost Max-Flow	Max flow with min cost	$O(F \times E \log V)$	Successive shortest paths, potentials
Hopcroft-Karp	Max bipartite matching	$O(E\sqrt{V})$	BFS + DFS phases
2-SAT	Boolean satisfiability	$O(n + C)$	Implication graph + SCC

Problem-Solving Strategy

- 1. Identify the problem type:** Read the problem statement carefully.
- 2. Check constraints:** If $V, E \leq 10^5$, use $O(V+E)$ or $O(E\sqrt{V})$ algorithms.
- 3. Choose the right tool:**
 - Directed graph with cycles → SCC
 - Undirected graph connectivity → Bridges/Articulation Points
 - Visit every edge → Euler path
 - Pairing/assignment → Bipartite Matching
 - Flow with capacities → Dinic
 - Flow with costs → MCMF
 - Boolean constraints → 2-SAT
- 4. Test edge cases:** Empty graph, single node, disconnected, etc.
- 5. Implement and test:** Use the provided templates.

End of Advanced Graph Algorithms Guide